



RAPPORT

Cryptographie

Réalisé Par :

- Mouhamadou Falilou Sarr

Sous la direction :

- Mr Fall

EXERCICE 1 :

1. Quelles sont les exigences fondamentales en sécurité informatique ?

Les exigences fondamentales en sécurité informatique comprennent la confidentialité, l'intégrité, la disponibilité, l'authenticité, la non-répudiation et la traçabilité des informations.

2. Citez quatre (4) services de sécurité et quatre (4) mécanismes de sécurité définis dans X.800 :

Les services de sécurité incluent l'authentification, l'autorisation, la confidentialité et l'intégrité. Les mécanismes de sécurité comprennent le chiffrement symétrique, le chiffrement asymétrique, les fonctions de hachage et les protocoles d'échange de clés.

3. Quelles sont les principales différences entre RSA et AES en termes de fonctionnalité et d'utilisation ?

RSA est un algorithme de cryptographie asymétrique utilisé pour l'authentification, la signature numérique et l'échange de clés. Il utilise une paire de clés (publique et privée) pour chiffrer et déchiffrer les données. Les clés RSA sont généralement plus grandes et le processus est plus lent que celui d'AES. AES, en revanche, est un algorithme de cryptographie symétrique utilisé principalement pour le chiffrement et le déchiffrement de données sensibles. Il utilise la même clé pour les deux opérations et est plus rapide et efficace que RSA. Les clés AES sont plus courtes que celles de RSA.

4. Expliquez le fonctionnement de l'algorithme RSA en détail, en incluant les étapes clés telles que la génération de clés, le chiffrement et le déchiffrement :

L'algorithme RSA (Rivest-Shamir-Adleman) est un algorithme de cryptographie asymétrique largement utilisé pour l'authentification, la signature numérique et le chiffrement. Voici comment il fonctionne en détail :

1. Génération des clés :

- Sélection de deux nombres premiers distincts, généralement désignés par p et q .

- Calcul du produit de ces deux nombres, $n = p * q$, qui constitue la partie publique de la clé.

- Calcul de la fonction d'Euler de n , notée $\phi(n) = (p - 1)(q - 1)$.
- Sélection d'un entier e , qui est relativement premier à $\phi(n)$ et inférieur à $\phi(n)$. Cet entier constitue la partie publique de la clé.
- Calcul de l'entier d , qui est l'inverse multiplicatif de e modulo $\phi(n)$. L'entier d constitue la partie privée de la clé.
- La clé publique est constituée de n et e , tandis que la clé privée est constituée de n et d .

2. Chiffrement :

- Pour chiffrer un message M en utilisant la clé publique (n, e) , le message est converti en un entier m tel que $0 \leq m < n$.
- Le message chiffré C est obtenu en calculant $C \equiv m^e \pmod{n}$.
- Le message chiffré C peut être envoyé en toute sécurité au destinataire.

3. Déchiffrement :

- Le destinataire reçoit le message chiffré C et utilise sa clé privée (n, d) pour le déchiffrer.
- Pour déchiffrer C , le destinataire calcule $m \equiv C^d \pmod{n}$.
- Le destinataire obtient ainsi l'entier m , qui est converti en texte clair pour récupérer le message M d'origine.

5. Expliquez le mode de fonctionnement AES-CBC avec un schéma illustratif :

Le mode de chiffrement AES-CBC (Cipher Block Chaining) utilise un schéma où chaque bloc de texte clair est combiné avec le bloc chiffré précédent avant le chiffrement. Cela crée un effet de chaînage qui augmente la sécurité du chiffrement. Chaque bloc chiffré dépend du bloc précédent, rendant le chiffrement plus sûr.

6. Comment la sécurité des courbes elliptiques est évaluée et comment elle se compare à celle de RSA ?

La sécurité des courbes elliptiques est évaluée en fonction de la taille des paramètres utilisés dans l'algorithme. Plus la taille est grande, plus il est difficile

pour une attaque de force brute de réussir. Les courbes elliptiques offrent une sécurité comparable à celle de RSA, mais avec des clés beaucoup plus courtes, ce qui les rend souvent plus efficaces en termes de performances et de stockage. La comparaison exacte dépend des paramètres spécifiques utilisés dans chaque algorithme et des avancées dans les méthodes d'attaque.

7. Quelles sont les propriétés des fonctions de hachage :

- Déterminisme : Pour une entrée donnée, la fonction de hachage doit toujours produire la même sortie.
- Rapide à calculer : La fonction de hachage doit être rapide à calculer pour ne pas ralentir le processus.
- Résistance à la pré-image : Il doit être difficile de trouver l'entrée à partir d'une sortie donnée.
- Résistance à la seconde pré-image : Il doit être difficile de trouver une deuxième entrée qui produit la même sortie.
- Résistance aux collisions : Il doit être difficile de trouver deux entrées différentes qui produisent la même sortie.

8. Décrivez les mécanismes de sécurité d'un réseau privé virtuel (VPN) utilisant le protocole IPSec.

Un VPN utilisant le protocole IPSec utilise deux principaux mécanismes de sécurité : l'Authentication Header (AH) et le Encapsulating Security Payload (ESP). AH assure l'intégrité et l'authenticité des paquets IP en ajoutant un en-tête qui contient un code d'authentification du message (MAC). ESP, quant à lui, fournit la confidentialité en chiffrant les données du paquet IP.

9. Comparez les modes de fonctionnement Transport et Tunnel d'IPSec.

Le mode Transport d'IPSec sécurise les données entre deux hôtes, tandis que le mode Tunnel sécurise les données entre deux passerelles VPN. Le mode Transport protège seulement le contenu du paquet IP, tandis que le mode Tunnel protège tout le paquet IP, y compris les en-têtes.

10. Quel est le rôle de IKE (Internet Key Exchange) dans IPSec ?

IKE est utilisé pour établir les paramètres de sécurité IPSec, notamment les clés de chiffrement et les méthodes d'authentification, entre les deux extrémités d'une connexion VPN.

11. Décrivez les principaux composants de IPSec : AH et ESP.

AH (Authentication Header) fournit l'intégrité et l'authenticité des paquets IP en utilisant des codes d'authentification de message (MAC). ESP (Encapsulating Security Payload) fournit la confidentialité des données en chiffrant les paquets IP.

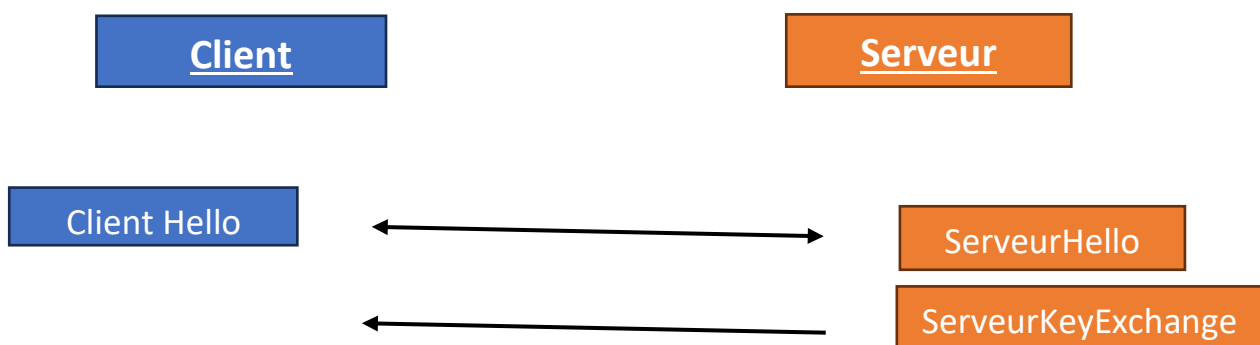
12. Qu'est-ce qu'une SA et à quoi sert-il dans le contexte de IPSec ?

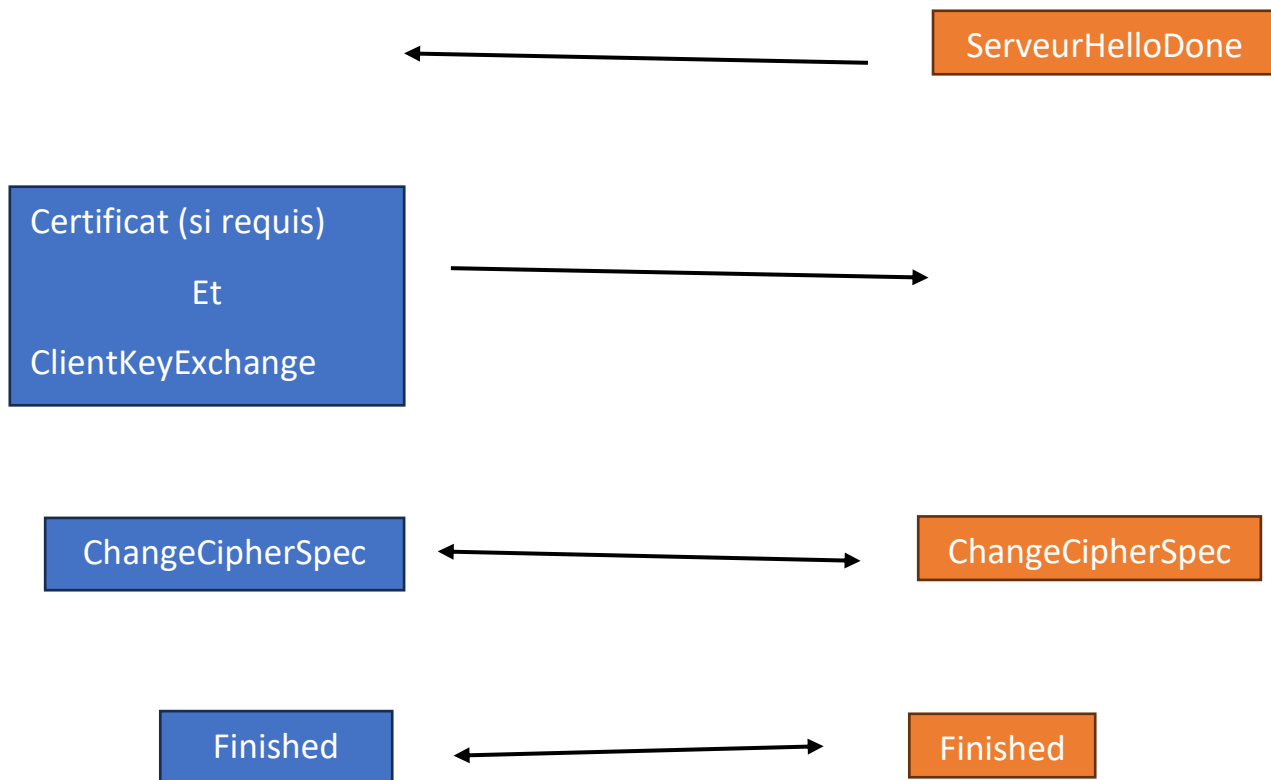
Une SA (Security Association) est un ensemble de paramètres de sécurité, tels que les algorithmes de chiffrement et d'authentification, associés à une communication unidirectionnelle entre deux entités IPSec. Il est utilisé pour sécuriser le trafic dans le contexte d'IPSec.

13. Quelles sont les différences entre SAD et SPD dans IPSec ?

SAD (Security Association Database) est une base de données qui stocke les informations spécifiques à chaque SA, telles que les clés et les paramètres de chiffrement. SPD (Security Policy Database) est une base de données qui définit les politiques de sécurité pour le traitement du trafic IP, telles que les règles de filtrage et les actions à prendre sur le trafic.

14. Faites un schéma explicatif du processus d'établissement d'une connexion sécurisée SSL/TLS entre un client et un serveur. Incluez les étapes de négociation de la sécurité, de vérification du certificat et d'échange de clés :





1. Le client envoie un message ClientHello contenant les paramètres de la connexion et les algorithmes de chiffrement qu'il supporte.
2. Le serveur répond avec un message ServerHello, qui contient les paramètres de la connexion choisis parmi ceux proposés par le client, ainsi qu'un certificat numérique contenant la clé publique du serveur.
3. Le serveur envoie un message ServerKeyExchange, qui contient des informations supplémentaires nécessaires à l'échange de clés si nécessaire.
4. Le serveur envoie un message ServerHelloDone pour indiquer qu'il a terminé la phase d'initialisation.
5. Le client vérifie le certificat du serveur, génère une clé de session, et envoie son propre certificat s'il est requis, ainsi que le message ClientKeyExchange.
6. Les deux parties échangent un message ChangeCipherSpec pour activer le chiffrement, puis envoient un message Finished pour confirmer que la connexion est établie de manière sécurisée.

15. Discutez des implications de l'évolution des technologies de cryptographie sur la sécurité des réseaux LAN, en tenant compte des avancées récentes telles que les algorithmes de chiffrement quantique :

L'évolution des technologies de cryptographie, notamment l'avènement des algorithmes de chiffrement quantique, présente des implications majeures pour la sécurité des réseaux LAN. Bien que les algorithmes quantiques offrent un potentiel de cryptographie inviolable face aux attaques des ordinateurs quantiques, leur adoption généralisée est entravée par des défis tels que la complexité de mise en œuvre et les coûts élevés. Pour garantir la sécurité à long terme, il est essentiel de maintenir l'utilisation de protocoles et d'algorithmes de cryptographie éprouvés, tout en explorant de nouvelles méthodes de cryptographie post-quantique

TP1 : LE HACHAGE

```
root@ubuntu: /home/falilou
GNU nano 4.8 message.txt
Ce message doit rester secret
```

1. Expliquez la différence entre le hachage, le chiffrement et la signature :

Le hachage, le chiffrement et la signature sont des techniques cryptographiques distinctes. Le hachage transforme une donnée en une empreinte unique, utile pour vérifier l'intégrité des données. Le chiffrement convertit les données en une forme illisible pour protéger leur confidentialité, tandis que la signature numérique utilise une clé privée pour créer une signature attachée à un message, vérifiable par quiconque possède la clé publique correspondante, assurant ainsi l'authenticité et l'intégrité du message. En bref, le hachage assure l'intégrité, le chiffrement la confidentialité, et la signature l'authentification et la non-répudiation.

2. Calculez le Hash (MD5, SHA256, SHA512) du message avec :

 Openssl :

- MD5 :

```
root@ubuntu:/home/falilou# nano message.txt
root@ubuntu:/home/falilou# openssl md5 message.txt
MD5(message.txt)= c8fef1f4cdb3731845abba9181ba0b78
root@ubuntu:/home/falilou#
```

- SHA256 :

```
root@ubuntu:/home/falilou# openssl sha256 message.txt
SHA256(message.txt)= 7755967c2762216d7973f9dd4640b322881fcd172470f400a93b81e5a3bfb1a1
root@ubuntu:/home/falilou#
```

- SHA512 :

```
root@ubuntu:/home/falilou# openssl sha512 message.txt
SHA512(message.txt)= 6770887b9f2f5ec7e7e4420e67766f2c4715c49882d598e1a55716a7d9c206107f6d1485ca2d5c1a509a0e9bb81b5032a8903add5f199231c5282ffe0b1cfc58
root@ubuntu:/home/falilou#
```

-  MD5 :

```
root@ubuntu:/home/falilou# md5sum message.txt
c8fef1f4cdb3731845abba9181ba0b78 message.txt
root@ubuntu:/home/falilou#
```

-  Certutil :

- MD5 :

```
C:\Users\sarrf>echo ce message doit rester secret > message.txt

C:\Users\sarrf>certutil -hashfile "message.txt" MD5
Hachage MD5 de message.txt :
6d703833a06fd09fe124174bacec8800
CertUtil: -hashfile La commande s'est terminée correctement.

C:\Users\sarrf>
```

- SHA256 :

```
C:\Users\sarrf>certutil -hashfile "message.txt" SHA256
Hachage SHA256 de message.txt :
41ec9c9672b4bc2f8fea8532705189a42b3162cf21078a7157064d0b723eeceb
CertUtil: -hashfile La commande s'est terminée correctement.

C:\Users\sarrf>
```

- SHA512 :

```
C:\Users\sarrf>certutil -hashfile "message.txt" SHA512
Hachage SHA512 de message.txt :
a03fa435211f8dcaa76e4eb1eb246f74e507f71f3cf17dce5f145d6082dcf613ec0a208816f025ab6a6c474082990cb9f92e3219277ecc9cdef74da3f22913aa
CertUtil: -hashfile La commande s'est terminée correctement.

C:\Users\sarrf>
```

- MD5 :

```
PS C:\Users\sarrf> Get-FileHash message.txt -Algorithm MD5
```

Algorithm	Hash	Path
MD5	6D703833A06FD09FE124174BACEC8800	C:\Users\sarrf\message.txt

```
PS C:\Users\sarrf>
```

- SHA256 :

```
PS C:\Users\sarrf> Get-FileHash message.txt -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	41EC9C9672B4BC2F8FEA8532705189A42B3162CF21078A7157064D0B723EECEB	C:\Users\sarrf\message.txt

```
PS C:\Users\sarrf>
```

- SHA512 :

```
PS C:\Users\sarrf> Get-FileHash message.txt -Algorithm SHA512
```

Algorithm	Hash	Path
SHA512	A03FA435211F8DCAA76E4EB1EB246F74E507F71F3CF17DCE5F145D6082DCF613EC0...	C:\Users\sarrf\message.txt

```
PS C:\Users\sarrf> |
```

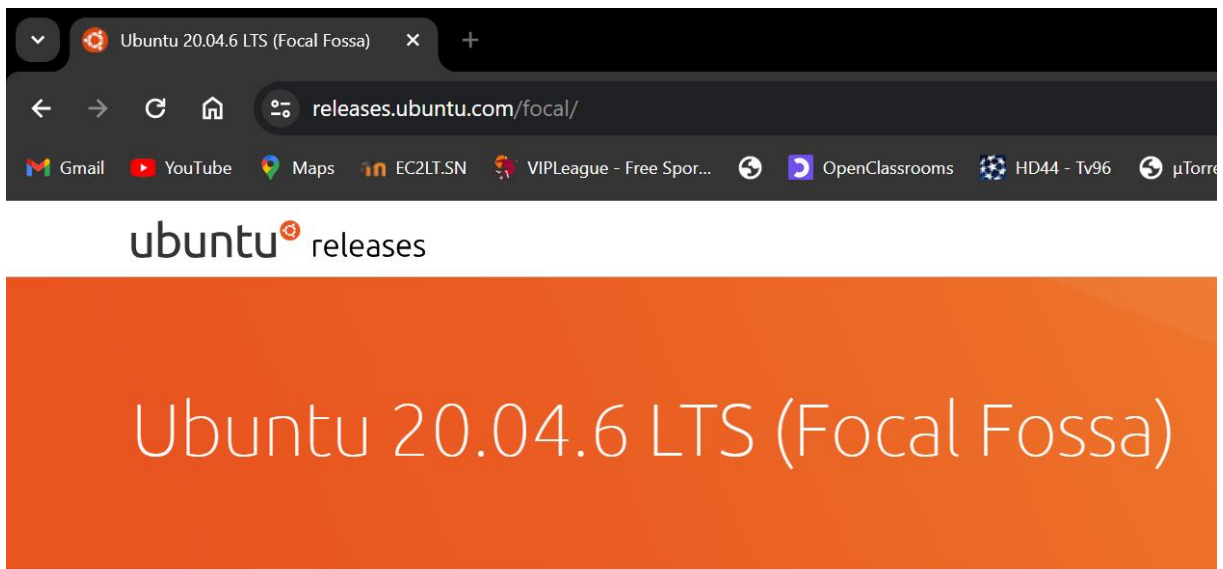
3. Combien de temps faudrait-il, en moyenne, pour qu'une collision se produise dans une fonction de hachage, connaissant la taille de l'espace des hachages possibles exprimée en bits ?

Le temps moyen pour qu'une collision se produise dans une fonction de hachage dépend de la taille de l'espace des hachages possibles exprimée en bits et du nombre de hash générés. En utilisant l'anniversaire paradoxal, on estime que le nombre d'opérations nécessaires pour qu'une collision se produise est d'environ la racine carrée de la taille de l'espace des hachages possibles. Pour calculer le temps moyen, on peut utiliser la formule suivante :

$$\text{Temps moyen} = \frac{2^{\text{nombre de bits}}}{2 \times \text{nombre de hash générés par seconde} \times 365 \times 24 \times 60 \times 60}$$

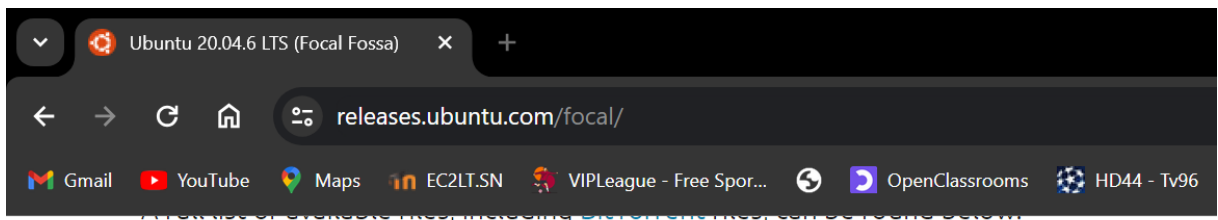
4. Téléchargez et Vérifiez l'intégrité d'une image iso Ubuntu 20 :

Téléchargeons l'image Ubuntu 20 :



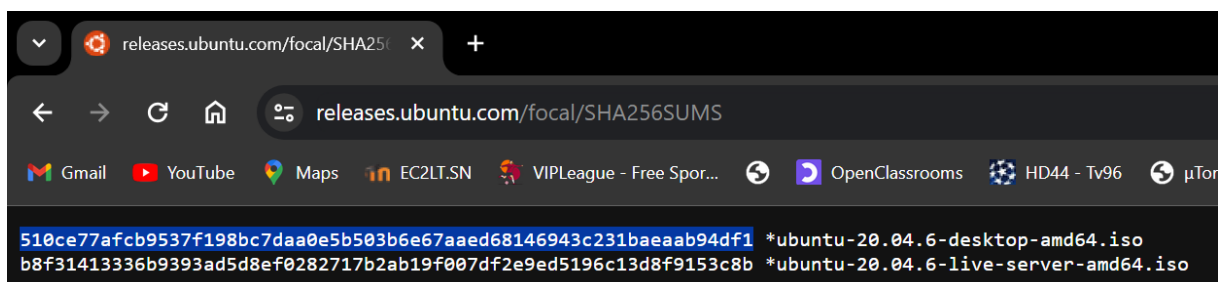
Select an image

On clique sur fichier de hachage l'image Ubuntu 20 et on copie la valeur de hachage :



If you need help burning these images to disk, see the [Image Burning Guide](#).

Name	Last modified	Size
 Parent Directory		-
 SHA256SUMS	2023-03-22 14:31	202



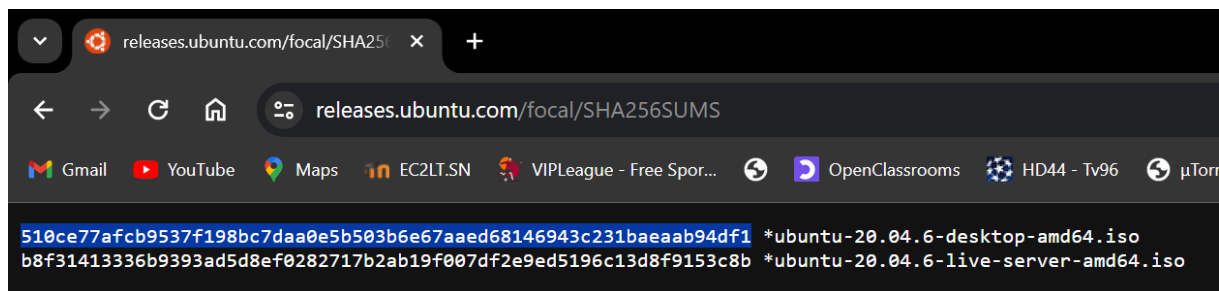


Vérifions l'intégrité de l'image ISO en exécutant la commande
CertUtil -hashfile "ubuntu-20.04.6-desktop-amd64.iso" SHA256 :

```
PS C:\Users\sarrf\Downloads> CertUtil -hashfile "ubuntu-20.04.6-desktop-amd64.iso" SHA256
Hachage SHA256 de ubuntu-20.04.6-desktop-amd64.iso :
510ce77afcb9537f198bc7daa0e5b503b6e67aaed68146943c231baeaab94df1
CertUtil: -hashfile La commande s'est terminée correctement.
PS C:\Users\sarrf\Downloads>
```



Comparons la valeur de hachage SHA256 affichée par la
commande avec la valeur de hachage correspondante dans le fichier
SHA256SUMS :



```
PS C:\Users\sarrf\Downloads> CertUtil -hashfile "ubuntu-20.04.6-desktop-amd64.iso" SHA256
Hachage SHA256 de ubuntu-20.04.6-desktop-amd64.iso :
510ce77afcb9537f198bc7daa0e5b503b6e67aaed68146943c231baeaab94df1
CertUtil: -hashfile La commande s'est terminée correctement.
PS C:\Users\sarrf\Downloads>
```

Les deux valeurs sont identiques !

TP2 : Chiffrement et signature avec l'algorithme RSA

Partie 1 : Chiffrement et déchiffrement

1. Générons une paire de clés RSA de 4096 bits avec openssl au format PEM :



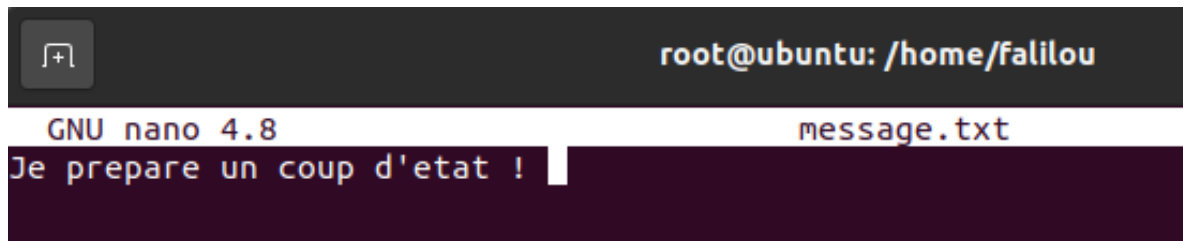
Générons la clé privée avec la commande : **openssl genrsa -out private_key.pem 4096**

```
root@ubuntu:/home/falilou# openssl genrsa -out private_key.pem 4096
Generating RSA private key, 4096 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
root@ubuntu:/home/falilou#
```

Générons la clé publique avec la commande : **openssl rsa -pubout -in private_key.pem -out public_key.pem**

```
root@ubuntu:/home/falilou# openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key
root@ubuntu:/home/falilou#
```

2. Créons le fichier message.txt avec un message en clair :



3. Chiffrons le message avec la clé publique avec la commande : **openssl rsautl -encrypt -pubin -inkey public_key.pem -in message.txt -out message_chiffre.txt**

```
root@ubuntu:/home/falilou# openssl rsautl -encrypt -pubin -inkey public_key.pem
-in message.txt -out message_chiffre.txt
root@ubuntu:/home/falilou#
```

4. Affichons le message chiffré :

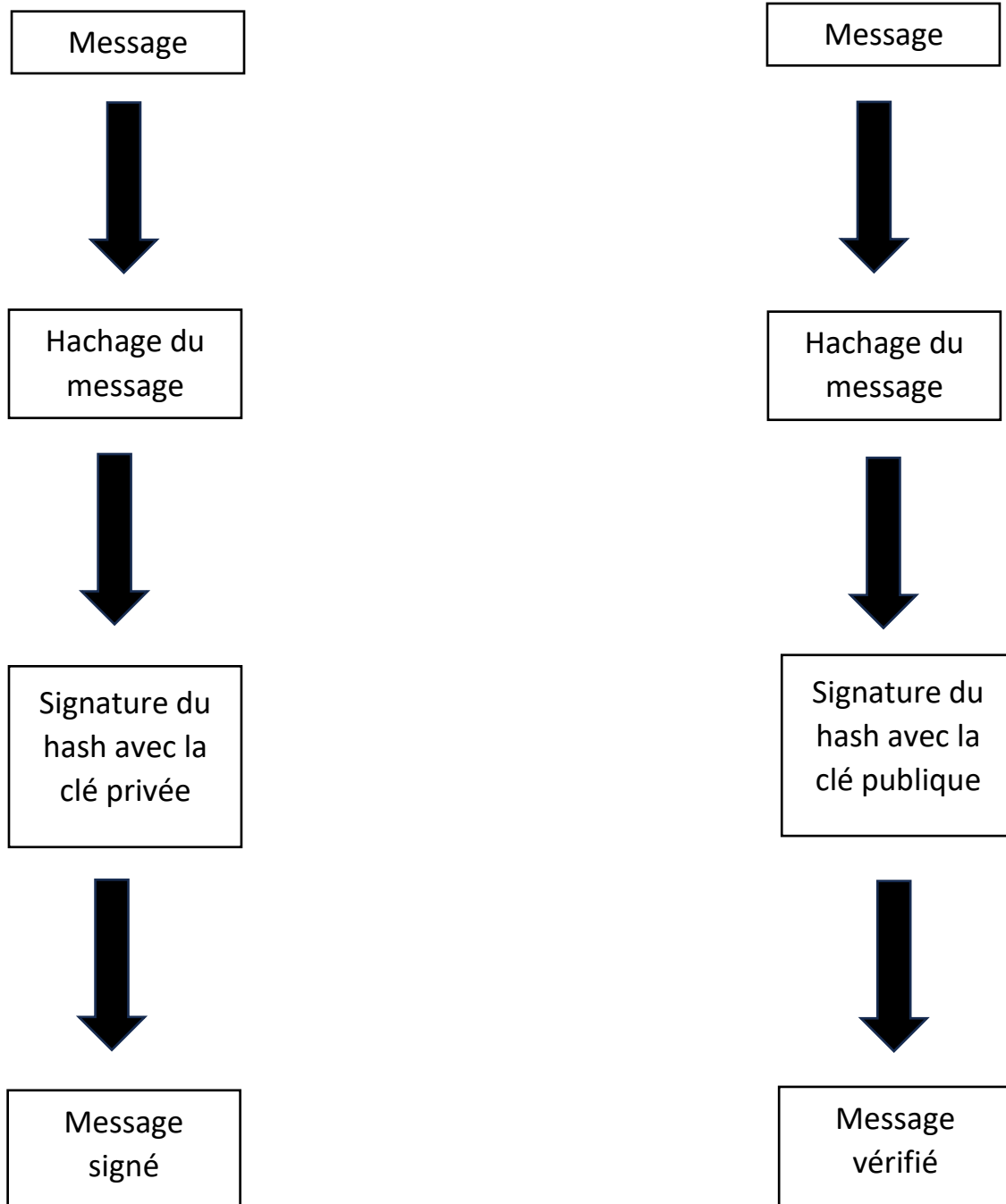


5. Déchiffrons le message avec la clé privée avec la commande : **openssl rsautl -decrypt -inkey private_key.pem -in message_chiffre.txt**

```
root@ubuntu:/home/falilou# openssl rsautl -decrypt -inkey private_key.pem -in message_chiffre.txt
Je prepare un coup d'etat !
root@ubuntu:/home/falilou#
```

Partie 2 : Signature

1. Faisons un schéma explicatif de la signature numérique :



2. Signons un message :



Créons le fichier avec notre message :

```
root@ubuntu: /home/falilou/Bureau
GNU nano 4.8 message.txt
Essayons de signer ce message !
```

 Générons le hash du message et copions le dans un fichier avec la commande : **openssl dgst -sha256 -binary < message.txt > message.sha256**

```
root@ubuntu:/home/falilou/Bureau# openssl dgst -sha256 -binary < message.txt > message.sha256
root@ubuntu:/home/falilou/Bureau#
```

 Utilisons la clé privée signer le message avec la commande : **openssl rsautl -sign -inkey private_key.pem -in message.sha256 -out signature.sha256**

```
root@ubuntu:/home/falilou/Bureau# openssl rsautl -sign -inkey private_key.pem -in message.sha256
-out signature.sha256
root@ubuntu:/home/falilou/Bureau#
```

3. Vérifions le message :

 Utilisons la clé publique pour déchiffrer le hash générer et vérifier la signature avec la commande : **openssl rsautl -verify -inkey public_key.pem -pubin -in signature.sha256 -out signature_dechiffre.sha256**

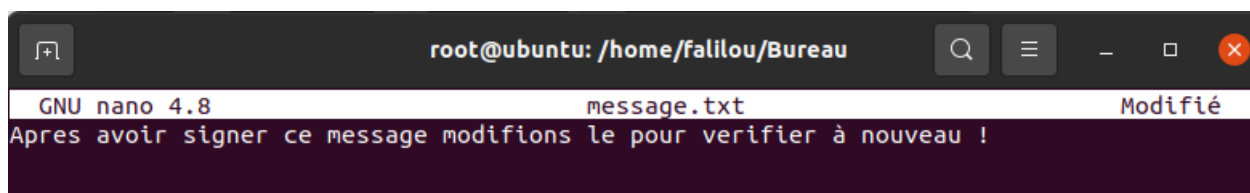
```
root@ubuntu:/home/falilou/Bureau# openssl rsautl -verify -inkey public_key.pem -pubin
-in signature.sha256 -out signature_dechiffre.sha256
root@ubuntu:/home/falilou/Bureau#
```

 Comparons les deux hash à l'aide de la commande : **diff -s message.sha256 signature_dechiffre.sha256**

```
root@ubuntu:/home/falilou/Bureau# diff -s message.sha256 signature_dechiffre.sha256
Les fichiers message.sha256 et signature_dechiffre.sha256 sont identiques
root@ubuntu:/home/falilou/Bureau#
```

4. Modifions le message et revérifions la signature :

 Modifions le message dans le fichier message.txt :



 Générons le nouveau hash du message avec la commande : **openssl dgst -sha256 -binary < message.txt > new_message_sha256**

```
root@ubuntu:/home/falilou/Bureau# openssl dgst -sha256 -binary < message.txt > new_message_sha256
root@ubuntu:/home/falilou/Bureau#
```



Signons à nouveau le message avec la clé privée à l'aide de la commande : **openssl rsautl -sign -inkey private_key.pem -in new_message.sha256 -out new_signature.sha256**

```
root@ubuntu:/home/falilou/Bureau# openssl rsautl -sign -inkey private_key.pem -in new_message.sha256 -out new_signature.sha256
root@ubuntu:/home/falilou/Bureau#
```



Comparons les deux hash à l'aide de la commande : **diff -s signature.sha256 new_signature.sha256**

```
root@ubuntu:/home/falilou/Bureau# diff -s signature.sha256 new_signature.sha256
Les fichiers binaires signature.sha256 et new_signature.sha256 sont différents
root@ubuntu:/home/falilou/Bureau#
```

5. Que constatez-vous ?

On constate que les deux hash sont différents par ce que le message a été modifié. Et donc toute altération du message invalide la signature associée à ce message.

Partie 3 : Les courbes elliptiques

1. Listez les courbes elliptiques supportées par openssl avec la commande **openssl ecparam -list_curves** :

```
root@ubuntu:/home/falilou/Bureau# openssl ecparam -list_curves
secp112r1 : SECG/WTLS curve over a 112 bit prime field
secp112r2 : SECG curve over a 112 bit prime field
secp128r1 : SECG curve over a 128 bit prime field
secp128r2 : SECG curve over a 128 bit prime field
secp160k1 : SECG curve over a 160 bit prime field
secp160r1 : SECG curve over a 160 bit prime field
secp160r2 : SECG/WTLS curve over a 160 bit prime field
secp192k1 : SECG curve over a 192 bit prime field
secp224k1 : SECG curve over a 224 bit prime field
secp224r1 : NIST/SECG curve over a 224 bit prime field
secp256k1 : SECG curve over a 256 bit prime field
secp384r1 : NIST/SECG curve over a 384 bit prime field
secp521r1 : NIST/SECG curve over a 521 bit prime field
prime192v1 : NIST/X9.62/SECG curve over a 192 bit prime field
prime192v2 : X9.62 curve over a 192 bit prime field
prime192v3 : X9.62 curve over a 192 bit prime field
prime239v1 : X9.62 curve over a 239 bit prime field
prime239v2 : X9.62 curve over a 239 bit prime field
prime239v3 : X9.62 curve over a 239 bit prime field
prime256v1 : X9.62/SECG curve over a 256 bit prime field
sect113r1 : SECG curve over a 113 bit binary field
sect113r2 : SECG curve over a 113 bit binary field
sect131r1 : SECG/WTLS curve over a 131 bit binary field
sect131r2 : SECG curve over a 131 bit binary field
sect163k1 : NIST/SECG/WTLS curve over a 163 bit binary field
sect163r1 : SECG curve over a 163 bit binary field
sect163r2 : NIST/SECG curve over a 163 bit binary field
sect193r1 : SECG curve over a 193 bit binary field
sect193r2 : SECG curve over a 193 bit binary field
sect233k1 : NIST/SECG/WTLS curve over a 233 bit binary field
sect233r1 : NIST/SECG/WTLS curve over a 233 bit binary field
```

```

sect239k1 : SECG curve over a 239 bit binary field
sect283k1 : NIST/SECG curve over a 283 bit binary field
sect283r1 : NIST/SECG curve over a 283 bit binary field
sect409k1 : NIST/SECG curve over a 409 bit binary field
sect409r1 : NIST/SECG curve over a 409 bit binary field
sect571k1 : NIST/SECG curve over a 571 bit binary field
sect571r1 : NIST/SECG curve over a 571 bit binary field
c2pnb163v1: X9.62 curve over a 163 bit binary field
c2pnb163v2: X9.62 curve over a 163 bit binary field
c2pnb163v3: X9.62 curve over a 163 bit binary field
c2pnb176v1: X9.62 curve over a 176 bit binary field
c2tnb191v1: X9.62 curve over a 191 bit binary field
c2tnb191v2: X9.62 curve over a 191 bit binary field
c2tnb191v3: X9.62 curve over a 191 bit binary field
c2pnb208w1: X9.62 curve over a 208 bit binary field
c2tnb239v1: X9.62 curve over a 239 bit binary field
c2tnb239v2: X9.62 curve over a 239 bit binary field
c2tnb239v3: X9.62 curve over a 239 bit binary field
c2pnb272w1: X9.62 curve over a 272 bit binary field
c2pnb304w1: X9.62 curve over a 304 bit binary field
c2tnb359v1: X9.62 curve over a 359 bit binary field
c2pnb368w1: X9.62 curve over a 368 bit binary field
c2tnb431r1: X9.62 curve over a 431 bit binary field
wap-wsg-idm-ecid-wtls1: WTLS curve over a 113 bit binary field
wap-wsg-idm-ecid-wtls3: NIST/SECG/WTLS curve over a 163 bit binary field
wap-wsg-idm-ecid-wtls4: SECG curve over a 113 bit binary field
wap-wsg-idm-ecid-wtls5: X9.62 curve over a 163 bit binary field
wap-wsg-idm-ecid-wtls6: SECG/WTLS curve over a 112 bit prime field
wap-wsg-idm-ecid-wtls7: SECG/WTLS curve over a 160 bit prime field
wap-wsg-idm-ecid-wtls8: WTLS curve over a 112 bit prime field
wap-wsg-idm-ecid-wtls9: WTLS curve over a 160 bit prime field
wap-wsg-idm-ecid-wtls10: NIST/SECG/WTLS curve over a 233 bit binary field
wap-wsg-idm-ecid-wtls11: NIST/SECG/WTLS curve over a 233 bit binary field
wap-wsg-idm-ecid-wtls12: WTLS curve over a 224 bit prime field
Oakley-EC2N-3:
    IPSec/IKE/Oakley curve #3 over a 155 bit binary field.
    Not suitable for ECDSA.
    Questionable extension field!
Oakley-EC2N-4:
    IPSec/IKE/Oakley curve #4 over a 185 bit binary field.
    Not suitable for ECDSA.
    Questionable extension field!
brainpoolP160r1: RFC 5639 curve over a 160 bit prime field
brainpoolP160t1: RFC 5639 curve over a 160 bit prime field
brainpoolP192r1: RFC 5639 curve over a 192 bit prime field
brainpoolP192t1: RFC 5639 curve over a 192 bit prime field
brainpoolP224r1: RFC 5639 curve over a 224 bit prime field
brainpoolP224t1: RFC 5639 curve over a 224 bit prime field
brainpoolP256r1: RFC 5639 curve over a 256 bit prime field
brainpoolP256t1: RFC 5639 curve over a 256 bit prime field
brainpoolP320r1: RFC 5639 curve over a 320 bit prime field
brainpoolP320t1: RFC 5639 curve over a 320 bit prime field
brainpoolP384r1: RFC 5639 curve over a 384 bit prime field
brainpoolP384t1: RFC 5639 curve over a 384 bit prime field
brainpoolP512r1: RFC 5639 curve over a 512 bit prime field
brainpoolP512t1: RFC 5639 curve over a 512 bit prime field
SM2       : SM2 curve over a 256 bit prime field
root@ubuntu:/home/falilou/Bureau# █

```

2. Créons une paire de clés cryptographiques basée sur les courbes elliptiques avec openssl :



Générons une clé de session avec la commande :

openssl ecparam -genkey -name prime256v1 -out ec_private_key.pem

```

root@ubuntu:/home/falilou/Bureau# openssl ecparam -genkey -name prime256v1 -out ec_private_key.pem
root@ubuntu:/home/falilou/Bureau# █

```

 Générons la clé publique avec la commande : **openssl ec -in ec_private_key.pem -pubout -out ec_public_key.pem**

```
root@ubuntu:/home/falilou/Bureau# openssl ec -in ec_private_key.pem -pubout -out ec_public_key.pem
read EC key
writing EC key
root@ubuntu:/home/falilou/Bureau#
```

 Générons une clé symétrique aléatoire avec la commande :
openssl rand -out session_key.bin 32

```
root@ubuntu:/home/falilou/Bureau# openssl rand -out session_key.bin 32
root@ubuntu:/home/falilou/Bureau#
```

3. Chiffrons/déchiffrons un message et signons/vérifions une signature avec la paire de clés et la clé symétrique aléatoires :

 Chiffrons le message avec la commande : **openssl enc -aes-256-cbc -salt -in message.txt -out encrypted_message.bin -pass file:session_key.bin**

```
root@ubuntu:/home/falilou/Bureau# openssl enc -aes-256-cbc -salt -in message.txt -out encrypted_message.bin -pass file:session_key.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ubuntu:/home/falilou/Bureau#
```

 Déchiffrons le message avec la commande : **openssl enc -d -aes-256-cbc -in encrypted_message.bin -out decrypted_message.txt -pass file:session_key.bin**

```
root@ubuntu:/home/falilou/Bureau# openssl enc -d -aes-256-cbc -in encrypted_message.bin -out decrypted_message.txt -pass file:session_key.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ubuntu:/home/falilou/Bureau# cat decrypted_message.txt
Après avoir signer ce message modifions le pour verifier à nouveau !
root@ubuntu:/home/falilou/Bureau#
```

 Signons le message avec la commande : **openssl dgst -sha256 -sign ec_private_key.pem -out message.sha256 message.txt**

```
root@ubuntu:/home/falilou/Bureau# openssl dgst -sha256 -sign ec_private_key.pem -out message.sha256 message.txt
root@ubuntu:/home/falilou/Bureau#
```

 Vérifions la signature du message avec la commande :
openssl dgst -sha256 -verify ec_public_key.pem -signature message.sha256 message.txt

```
root@ubuntu:/home/falilou/Bureau# openssl dgst -sha256 -verify ec_public_key.pem -signature message.sha256 message.txt
Verified OK
root@ubuntu:/home/falilou/Bureau#
```

TP3 : Chiffrement symétrique

1. Listons les algorithmes de chiffrement symétrique supportés par openssl avec la commande : **openssl list -cipher-algorithms**

```
root@ubuntu:/home/falilou/Bureau# openssl list -cipher-algorithms
AES-128-CBC
AES-128-CBC-HMAC-SHA1
AES-128-CBC-HMAC-SHA256
id-aes128-CCM
AES-128-CFB
AES-128-CFB1
AES-128-CFB8
AES-128-CTR
AES-128-ECB
id-aes128-GCM
AES-128-OCB
AES-128-OFB
AES-128-XTS
AES-192-CBC
id-aes192-CCM
AES-192-CFB
AES-192-CFB1
AES-192-CFB8
AES-192-CTR
AES-192-ECB
id-aes192-GCM
AES-192-OCB
AES-192-OFB
AES-256-CBC
AES-256-CBC-HMAC-SHA1
AES-256-CBC-HMAC-SHA256
id-aes256-CCM
AES-256-CFB
AES-256-CFB1
AES-256-CFB8
AES-256-CTR
AES-256-ECB
id-aes256-GCM
AES-256-OCB
AES-256-OFB
AES-256-XTS
aes128 => AES-128-CBC
aes128-wrap => id-aes128-wrap
aes192 => AES-192-CBC
aes192-wrap => id-aes192-wrap
aes256 => AES-256-CBC
aes256-wrap => id-aes256-wrap
ARIA-128-CBC
ARIA-128-CCM
ARIA-128-CFB
ARIA-128-CFB1
ARIA-128-CFB8
ARIA-128-CTR
ARIA-128-ECB
ARIA-128-GCM
ARIA-128-OFB
ARIA-192-CBC
ARIA-192-CCM
ARIA-192-CFB
ARIA-192-CFB1
ARIA-192-CFB8
ARIA-192-CTR
ARIA-192-ECB
ARIA-192-GCM
ARIA-192-OFB
ARIA-256-CBC
ARIA-256-CCM
ARIA-256-CFB
ARIA-256-CFB1
ARIA-256-CFB8
ARIA-256-CTR
ARIA-256-ECB
ARIA-256-GCM
ARIA-256-OFB
aria128 => ARIA-128-CBC
aria192 => ARIA-192-CBC
aria256 => ARIA-256-CBC
```

```

bf => BF-CBC
BF-CBC
BF-CFB
BF-ECB
BF-OFB
blowfish => BF-CBC
CAMELLIA-128-CBC
CAMELLIA-128-CFB
CAMELLIA-128-CFB1
CAMELLIA-128-CFB8
CAMELLIA-128-CTR
CAMELLIA-128-ECB
CAMELLIA-128-OFB
CAMELLIA-192-CBC
CAMELLIA-192-CFB
CAMELLIA-192-CFB1
CAMELLIA-192-CFB8
CAMELLIA-192-CTR
CAMELLIA-192-ECB
CAMELLIA-192-OFB
CAMELLIA-256-CBC
CAMELLIA-256-CFB
CAMELLIA-256-CFB1
CAMELLIA-256-CFB8
CAMELLIA-256-CTR
CAMELLIA-256-ECB
CAMELLIA-256-OFB
camellia128 => CAMELLIA-128-CBC
camellia192 => CAMELLIA-192-CBC
camellia256 => CAMELLIA-256-CBC
cast => CAST5-CBC
cast-cbc => CAST5-CBC
CAST5-CBC
CAST5-CFB
CAST5-ECB
CAST5-OFB
ChaCha20
ChaCha20-Poly1305
des => DES-CBC
DES-CBC
DES-CFB
DES-CFB1
DES-CFB8
DES-ECB
DES-EDE
DES-EDE-CBC
DES-EDE-CFB
des-ede-ecb => DES-EDE
DES-EDE-OFB
DES-EDE3
DES-EDE3-CBC
DES-EDE3-CFB
DES-EDE3-CFB1
DES-EDE3-CFB8
des-ede3-ecb => DES-EDE3
DES-EDE3-OFB
DES-OFB
des3 => DES-EDE3-CBC
des3-wrap => id-smime-alg-CMS3DESwrap
desx => DESX-CBC
DESX-CBC
id-aes128-CCM
id-aes128-GCM
id-aes128-wrap
id-aes128-wrap-pad
id-aes192-CCM
id-aes192-GCM
id-aes192-wrap
id-aes192-wrap-pad
id-aes256-CCM
id-aes256-GCM
id-aes256-wrap
id-aes256-wrap-pad
id-smime-alg-CMS3DESwrap
rc2 => RC2-CBC
rc2-128 => RC2-CBC
rc2-40 => RC2-40-CBC
RC2-40-CBC
rc2-64 => RC2-64-CBC

```

```

RC2-64-CBC
RC2-CBC
RC2-CFB
RC2-ECB
RC2-OFB
RC4
RC4-40
RC4-HMAC-MD5
seed => SEED-CBC
SEED-CBC
SEED-CFB
SEED-ECB
SEED-OFB
sm4 => SM4-CBC
SM4-CBC
SM4-CFB
SM4-CTR
SM4-ECB
SM4-OFB
root@ubuntu:/home/falilou/Bureau#

```

2. Différence entre la clé de chiffrement et l'algorithme de chiffrement dans la cryptographie symétrique :

La clé de chiffrement est une donnée secrète utilisée pour chiffrer et déchiffrer des données dans un algorithme de chiffrement symétrique, tandis que l'algorithme de chiffrement est la méthode ou le protocole utilisé pour effectuer le chiffrement et le déchiffrement. En d'autres termes, la clé de chiffrement est la "clé" réelle utilisée pour sécuriser les données, tandis que l'algorithme de chiffrement définit la façon dont cette clé est utilisée pour effectuer les opérations de chiffrement et de déchiffrement.

3. Différence entre une clé en tant que mot passe et une clé en tant que fichier dans le chiffrement AES avec openssl :

Dans le chiffrement AES avec OpenSSL, une clé peut être spécifiée soit en tant que mot de passe (par exemple via l'option `-pass`), soit en tant que fichier contenant la clé. Lorsqu'une clé est spécifiée en tant que mot de passe, OpenSSL utilise une dérivation de clé (comme PBKDF2) pour générer une clé à partir du mot de passe. Lorsqu'une clé est spécifiée en tant que fichier, OpenSSL lit simplement la clé à partir du fichier.

4. Créons un fichier contenant la clé secrète avec la commande : **openssl rand -out secret.key 32**

```

root@ubuntu:/home/falilou/Bureau# openssl rand -out secret.key 32
root@ubuntu:/home/falilou/Bureau#

```

5. Chiffons/déchiffons une photo avec cette clé en utilisant l'algorithme AES-256-CBC :

 Chiffons la photo avec la commande : **openssl enc -aes-256-cbc -salt -in download.jpg -out photo_chiffre.enc -pass file:secret.key**

```
root@ubuntu:/home/falilou/Bureau# openssl enc -aes-256-cbc -salt -in download.jpg -out photo_chiffre.enc -pass file:secret.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ubuntu:/home/falilou/Bureau#
```

 Déchiffons la photo avec la commande : **openssl enc -d -aes-256-cbc -in photo_chiffre.enc -out photo_dechiffre.jpg -pass file:secret.key**

```
root@ubuntu:/home/falilou/Bureau# openssl enc -d -aes-256-cbc -in photo_chiffre.enc -out photo_dechiffre.jpg -pass file:secret.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ubuntu:/home/falilou/Bureau#
```

6. Comment partager une clé secrète :

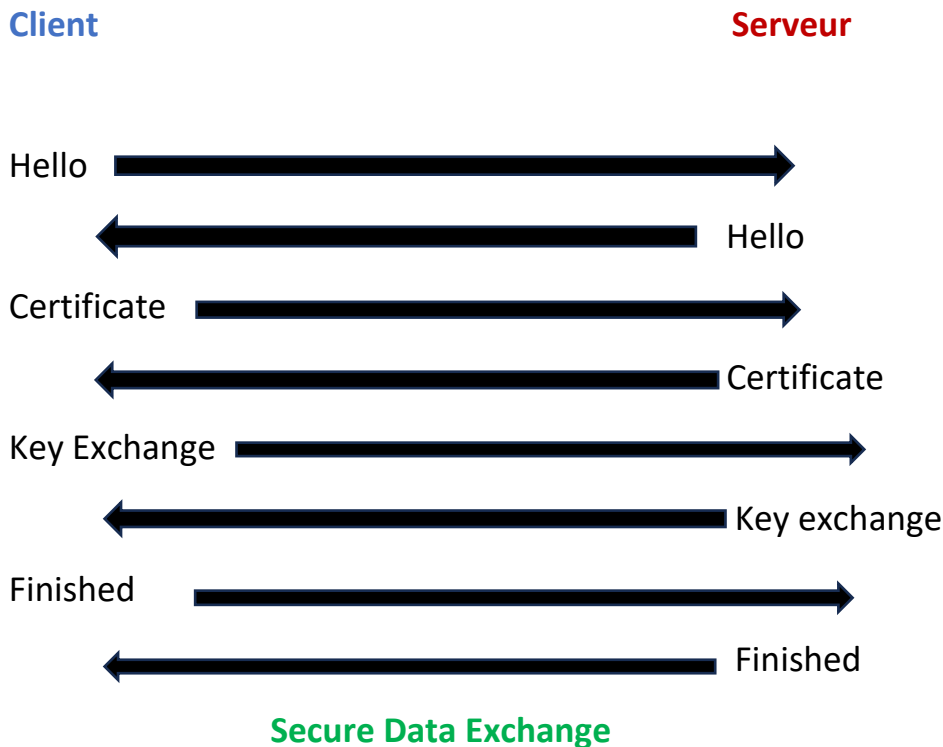
Il existe plusieurs méthodes pour partager une clé secrète de manière sécurisée :

- Canal sécurisé : On utilise un canal sécurisé tel que HTTPS, SFTP ou SCP pour transmettre la clé secrète. On s'assure que le canal de communication est chiffré et authentifié pour empêcher les interceptions et les attaques de l'homme du milieu.
- Chiffrement asymétrique : Utilisez la cryptographie asymétrique (par exemple, RSA) pour chiffrer la clé secrète avant de l'envoyer. La personne qui souhaite partager la clé peut chiffrer la clé secrète avec la clé publique du destinataire. Une fois chiffrée, la clé secrète peut être envoyée sur un canal non sécurisé sans compromettre sa sécurité. Le destinataire peut ensuite utiliser sa clé privée pour déchiffrer la clé secrète.
- Échange de clés Diffie-Hellman : Les deux parties peuvent utiliser le protocole de l'échange de clés Diffie-Hellman pour convenir d'une clé secrète commune sans jamais avoir besoin de transmettre cette clé sur le réseau. Le protocole permet à chaque partie de générer une clé publique et d'échanger ces clés publiques de manière sécurisée. À partir de ces clés publiques, une clé secrète commune peut être calculée de manière sécurisée.

TP4 : Fonctionnement de TLS

Partie 1 : Fonctionnement TLS

1. Expliquons le fonctionnement de TLS avec un schéma :



1. Hello (Bonjour) : Le client envoie un message "Hello" au serveur pour initier la connexion TLS.
2. Certificate (Certificat) : Le serveur envoie son certificat au client, prouvant son identité.
3. Key Exchange (Échange de clés) : Le client et le serveur échangent des clés secrètes qui seront utilisées pour chiffrer et déchiffrer les données.
4. Finished (Terminé) : Les deux parties s'échangent des messages "Finished" pour confirmer que la connexion est établie de manière sécurisée.
5. Secure Data Exchange (Échange de données sécurisé) : Une fois la connexion établie, les données peuvent être échangées de manière sécurisée entre le client et le serveur

2. Dans le contexte de TLS comment le client vérifie l'authenticité du serveur ?

Pour vérifier l'authenticité du serveur dans le cadre de TLS, le client vérifie les éléments suivants :

- ❖ L'identité du serveur, généralement vérifiée en comparant le nom de domaine du serveur avec celui indiqué dans le certificat du serveur.
- ❖ L'authenticité du certificat du serveur, en vérifiant sa signature numérique à l'aide de l'autorité de certification (CA) de confiance du client.
- ❖ La validité du certificat, en s'assurant qu'il n'a pas expiré et qu'il est émis pour le bon domaine.

3. Définition d'une suite cryptographique :

Une suite cryptographique est un ensemble d'algorithmes cryptographiques utilisés ensemble pour fournir des services de sécurité dans un protocole de communication sécurisée, comme TLS. Ces algorithmes incluent généralement des méthodes d'échange de clés, d'authentification, de chiffrement et d'intégrité des données.

4. Quelles sont les différentes parties essentielles d'une suite cryptographique :

- ❖ Les différentes parties essentielles d'une suite cryptographique sont :
L'algorithme d'échange de clés pour établir une clé de session sécurisée entre le client et le serveur.
- ❖ L'algorithme d'authentification pour vérifier l'identité des parties communiquant.
- ❖ L'algorithme de chiffrement symétrique pour chiffrer les données échangées entre le client et le serveur.
- ❖ L'algorithme d'authentification de message pour vérifier l'intégrité des données échangées.

5.a- Citons quelques exemples de suites cryptographiques et leurs composants:



Suite TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 :

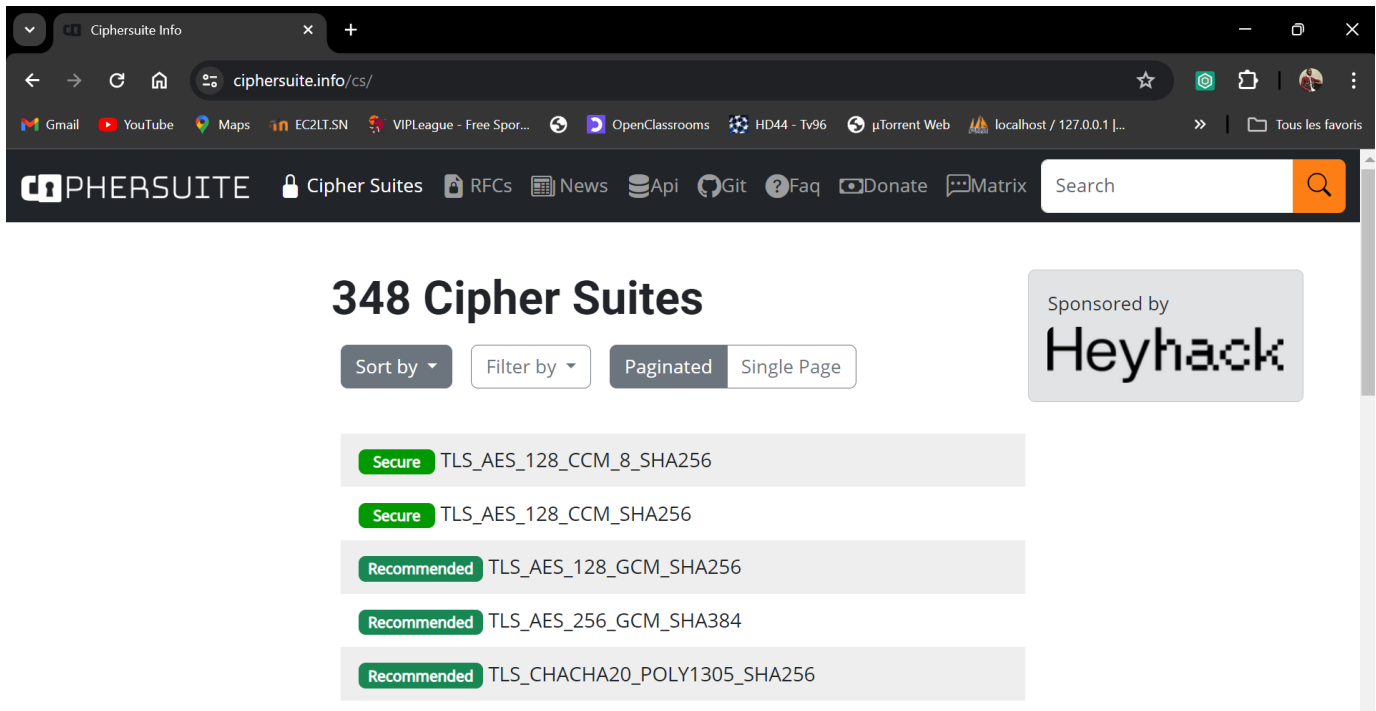
- Algorithme d'échange de clés : ECDHE (Ephemeral Elliptic Curve Diffie-Hellman)
- Algorithme d'authentification : RSA
- Algorithme de chiffrement symétrique : AES-256-GCM
- Algorithme d'authentification de message : SHA-384



Suite TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 :

- Algorithme d'échange de clés : DHE (Diffie-Hellman Ephemeral)
- Algorithme d'authentification : RSA
- Algorithme de chiffrement symétrique : AES-128-CBC
- Algorithme d'authentification de message : SHA-256

5.b- Allez sur le site <https://ciphersuite.info/> pour voir les suites faibles, sûres, et recommandées :



Partie 2 : Applications (mkcert+Apache2)

1. Installons les prérequis :

```
root@ubuntu:/home/falilou/Bureau# apt-get install wget libnss3-tools
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
wget est déjà la version la plus récente (1.20.3-1ubuntu2).
wget passé en « installé manuellement ».
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  linux-headers-5.15.0-94-generic linux-headers-5.4.0-173 linux-hwe-5.15-headers-5.15
  mobile-broadband-provider-info
Veuillez utiliser « apt autoremove » pour les supprimer.
Les NOUVEAUX paquets suivants seront installés :
  libnss3-tools
0 mis à jour, 1 nouvellement installés, 0 à enlever et 29 non mis à jour.
Il est nécessaire de prendre 1055 ko dans les archives.
Après cette opération, 5199 ko d'espace disque supplémentaires seront utilisés.
Souhaitez-vous continuer ? [O/n]
```

2. Téléchargeons l'outil mkcert :

wget https://github.com/FiloSottile/mkcert/releases/download/v1.4.3/mkcert-v1.4.3-linux-amd64

```
root@ubuntu:/home/falilou/Bureau# wget https://github.com/FiloSottile/mkcert/releases/download/v1.4.3/mkcert-v1.4.3-linux-amd64
--2024-05-06 13:20:08-- https://github.com/FiloSottile/mkcert/releases/download/v1.4.3/mkcert-v1.4.3-linux-amd64
Résolution de github.com (github.com)... 140.82.121.3
Connexion à github.com (github.com) [140.82.121.3]:443... connecté.
requête HTTP transmise, en attente de la réponse, 302 Found
Enplacement : https://objects.githubusercontent.com/github-production-release-asset-2e65be/138547797/89a9d600-2f20-11eb-9474-a9aab8fb1873?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAVCODYLSA53PQK42A92F20240506%2Fus-east-1%2F%3Faws4_request&X-Amz-Date=20240506T132008Z&X-Amz-Expires=300&X-Amz-Signature=b4fbd7691a1e43ed5a0f7b2cecb4ba8f85aeb025dc4a9e9f7c915dccb0bb85&X-Amz-SignedHeaders=host&actor_id=0&key_id=0&repo_id=138547797&response-content-disposition=attachment%3B%20filename%3Dmkcert-v1.4.3-linux-amd64&response-content-type=application%2Foctet-stream [suivant]
--2024-05-06 13:20:08-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/138547797/89a9d600-2f20-11eb-9474-a9aab8fb1873?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAVCODYLSA53PQK42A92F20240506%2Fus-east-1%2F%3Faws4_request&X-Amz-Date=20240506T132008Z&X-Amz-Expires=300&X-Amz-Signature=b4fbd7691a1e43ed5a0f7b2cecb4ba8f85aeb025dc4a9e9f7c915dccb0bb85&X-Amz-SignedHeaders=host&actor_id=0&key_id=0&repo_id=138547797&response-content-disposition=attachment%3B%20filename%3Dmkcert-v1.4.3-linux-amd64&response-content-type=application%2Foctet-stream
Résolution de objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connexion à objects.githubusercontent.com (objects.githubusercontent.com) [185.199.108.133]:443... connecté.
requête HTTP transmise, en attente de la réponse, 200 OK
Taille : 4803796 (4,6M) [application/octet-stream]
Enregistre : «mkcert-v1.4.3-linux-amd64»

mkcert-v1.4.3-linux-amd64 100%[=====] 4,58M 3,17MB/s ds 1,4s
2024-05-06 13:20:11 (3,17 MB/s) - «mkcert-v1.4.3-linux-amd64» enregistré [4803796/4803796]
root@ubuntu:/home/falilou/Bureau#
```

3. Déplaçons et renommons le fichier :

```
root@ubuntu:/home/falilou/Bureau# chmod +x /usr/bin/mkcert
root@ubuntu:/home/falilou/Bureau# mkcert --version
v1.4.3
root@ubuntu:/home/falilou/Bureau#
```

4. Affichez le dossier contenant le certificat et la clé privée de l'autorité avec un utilisateur simple :

```
falilou@ubuntu:~$ mkcert -CAROOT
/home/falilou/.local/share/mkcert
falilou@ubuntu:~$
```

5. Avec un utilisateur simple créons un certificat et une clé privée pour le serveur apache pour les domaines localhost et 127.0.0.1 :

Dans votre cas, vous remplacerez 127.0.0.1 par l'adresse IP de votre machine (c'est obligatoire).

```
falilou@ubuntu:~$ mkcert localhost 192.168.2.182
Created a new local CA 🌟
Note: the local CA is not installed in the system trust store.
Note: the local CA is not installed in the Firefox and/or Chrome/Chromium trust store.
Run "mkcert -install" for certificates to be trusted automatically ⚠️
Created a new certificate valid for the following names 📄
- "localhost"
- "192.168.2.182"
The certificate is at "./localhost+1.pem" and the key at "./localhost+1-key.pem"
✅
It will expire on 6 August 2026 📅
falilou@ubuntu:~$
```

6. Avec un utilisateur simple installons le certificat de l'autorité local (rootCA.pem) dans le magasin des autorités de certification racines de confiance du système :

```
falilou@ubuntu:~$ mkcert -install
The local CA is now installed in the system trust store! ⚡
The local CA is now installed in the Firefox and/or Chrome/Chromium trust store
(requires browser restart)! 🦊
falilou@ubuntu:~$
```

7. Nous remarquons que la clé privée et le certificat du serveur (qui vont être spécifiés dans le site virtuel) sont différents de ceux de l'autorité :

```
falilou@ubuntu:~$ ls localhost+1.pem localhost+1-key.pem
localhost+1-key.pem  localhost+1.pem
falilou@ubuntu:~$
```

8. Créons un site virtuel ssl en copiant le fichier de configuration par défaut :

```
root@ubuntu:/home/falilou# cp /etc/apache2/sites-available/default-ssl.conf
/etc/apache2/sites-available/falilou-ssl.conf
root@ubuntu:/home/falilou#
```

9. Renseignons les paramètres minimaux pour l'activation de https pour le site en question :

```
GNU nano 4.8 /etc/apache2/sites-available/falilou-ssl.conf
<IfModule mod_ssl.c>
  <VirtualHost _default_:443>
    ServerAdmin webmaster@localhost
    ServerName localhost
    DocumentRoot /var/www/html

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    SSLEngine on
    SSLCertificateFile      /home/falilou/localhost+1.pem
    SSLCertificateKeyFile  /home/falilou/localhost+1-key.pem

    <FilesMatch "\.(cgi|shtml|phtml|php)$">
      SSLOptions +StdEnvVars
    </FilesMatch>
    <Directory /usr/lib/cgi-bin>
      SSLOptions +StdEnvVars
    </Directory>
  </VirtualHost>
</IfModule>
```

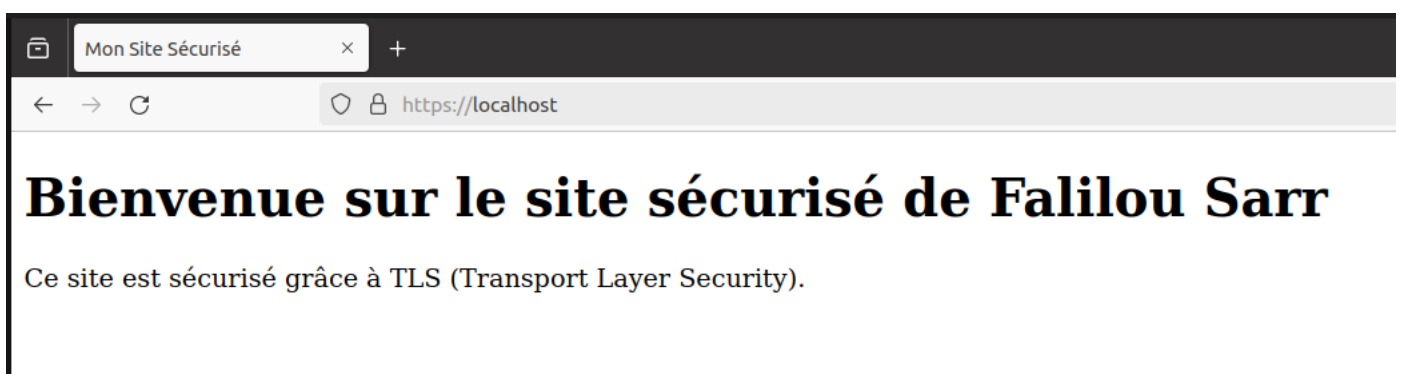
10. Activons le site virtuel et le module ssl :

```
root@ubuntu:/home/falilou# a2ensite falilou-ssl.conf
Enabling site falilou-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2
root@ubuntu:/home/falilou# a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
root@ubuntu:/home/falilou# systemctl reload apache2
root@ubuntu:/home/falilou#
```

11. Créons le fichier index de notre site web :

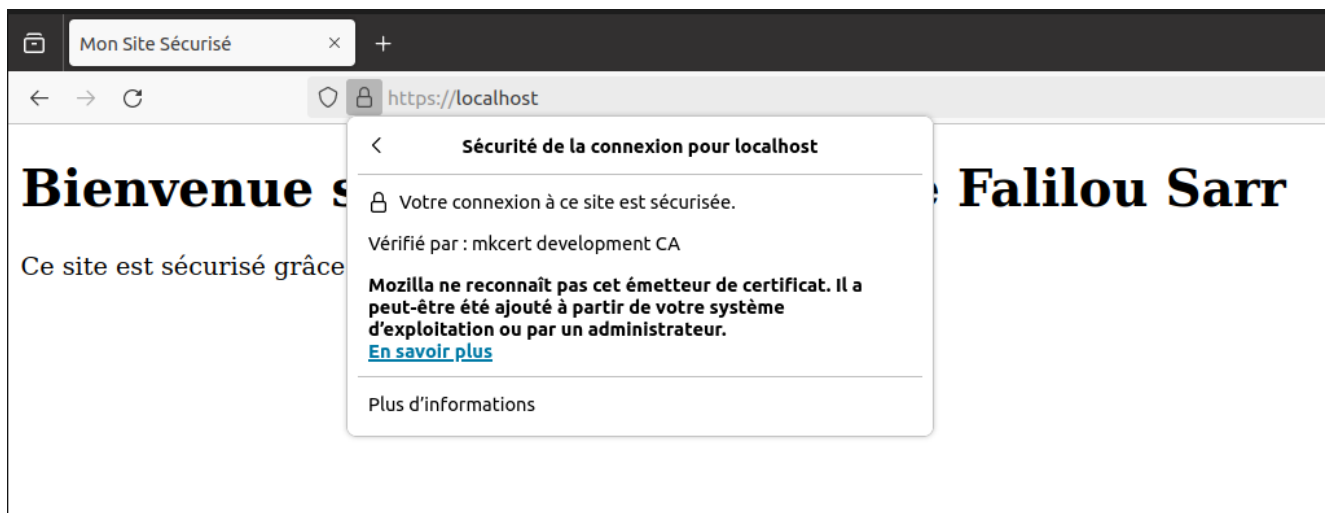
```
GNU nano 4.8 index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon Site Sécurisé</title>
</head>
<body>
  <h1>Bienvenue sur le site sécurisé de Falilou Sarr</h1>
  <p>Ce site est sécurisé grâce à TLS (Transport Layer Security).</p>
</body>
</html>
```

12. Utilisons https://localhost pour Accéder à notre site :

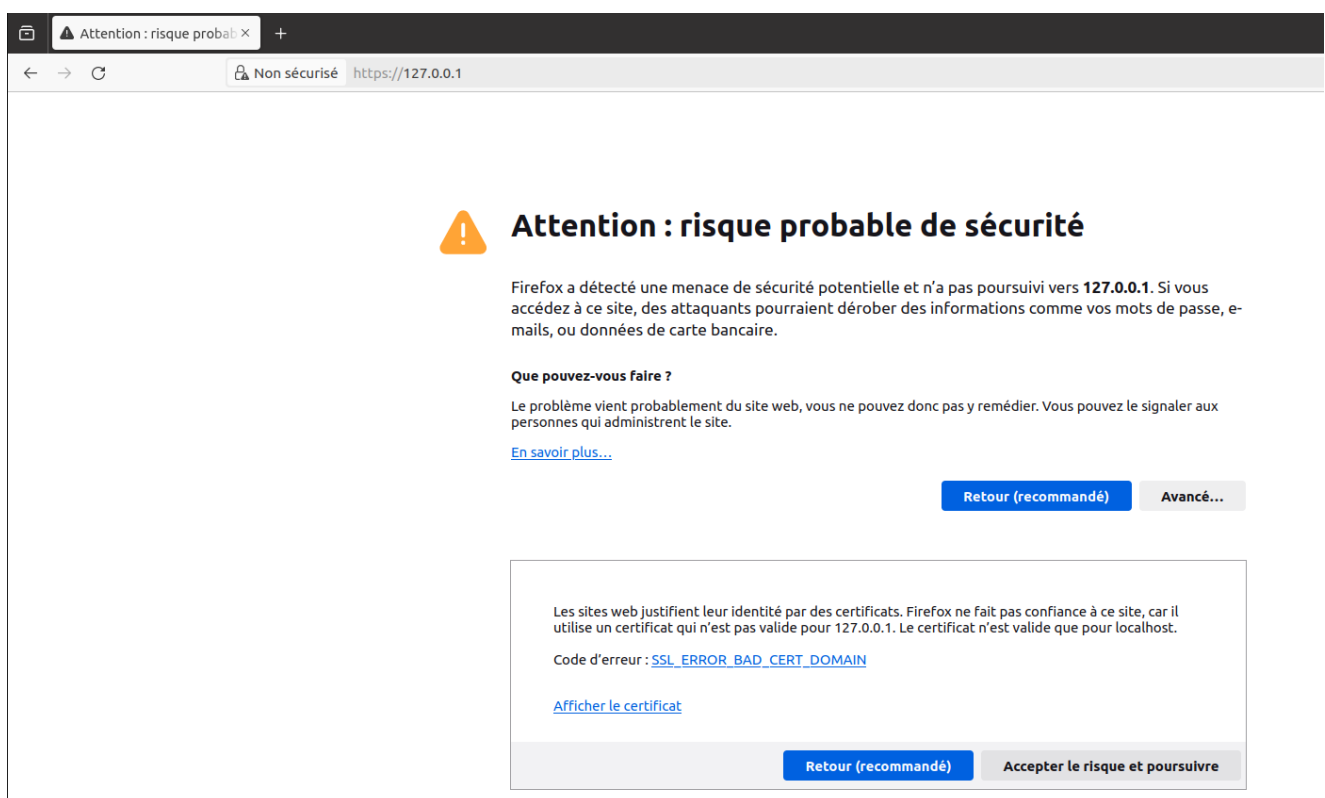


En utilisant HTTPS://localhost pour accéder au site, nous ne verrons pas l'exception de sécurité car mkcert crée des certificats auto-signés mais les installe également dans le magasin de certificats de confiance de votre système d'exploitation. Donc il n'affiche pas d'erreur de sécurité.

13. Sur le cadenas il n'y a pas aussi le point d'exclamation :



14. Remplaçons localhost par 127.0.0.1 et faisons le constat :

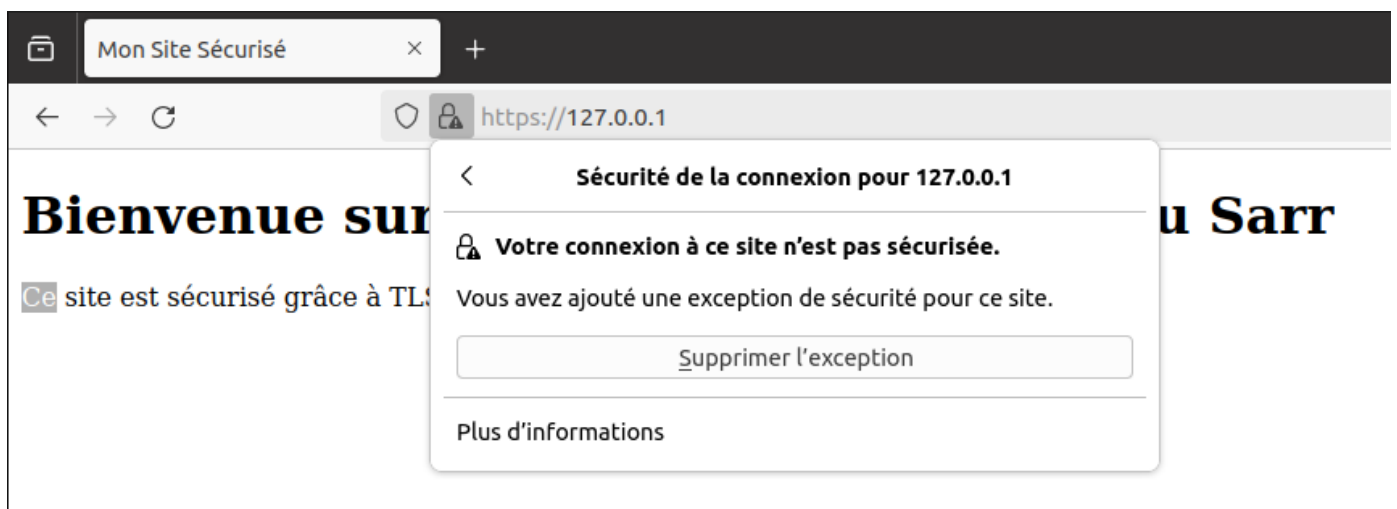


Lorsqu'on remplace localhost par 127.0.0.1 dans l'URL, On remarque qu'il y'a des messages d'avertissement liés à la sécurité. Car le certificat généré par mkcert ne correspond plus au domaine dans l'URL, Le certificat n'est valide que pour localhost. Cela entraîne une erreur car le certificat ne correspond pas au domaine spécifié dans la barre d'adresse du navigateur.

15. Dans ce cas de test on voit l'erreur **ssl_error_bad_cert_domain**. Qu'est-ce cela veut dire :

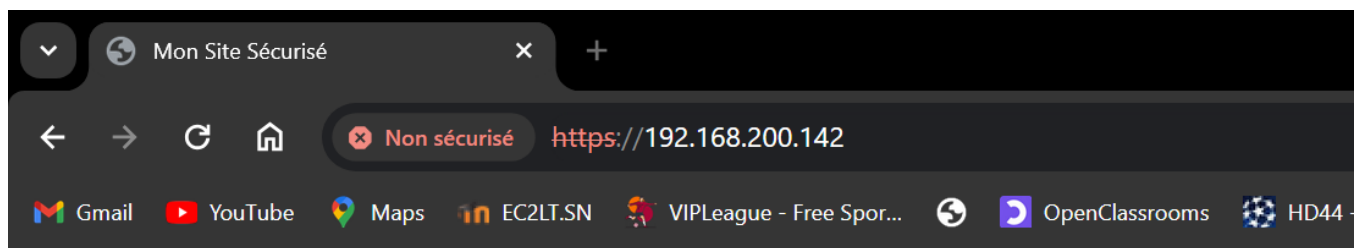
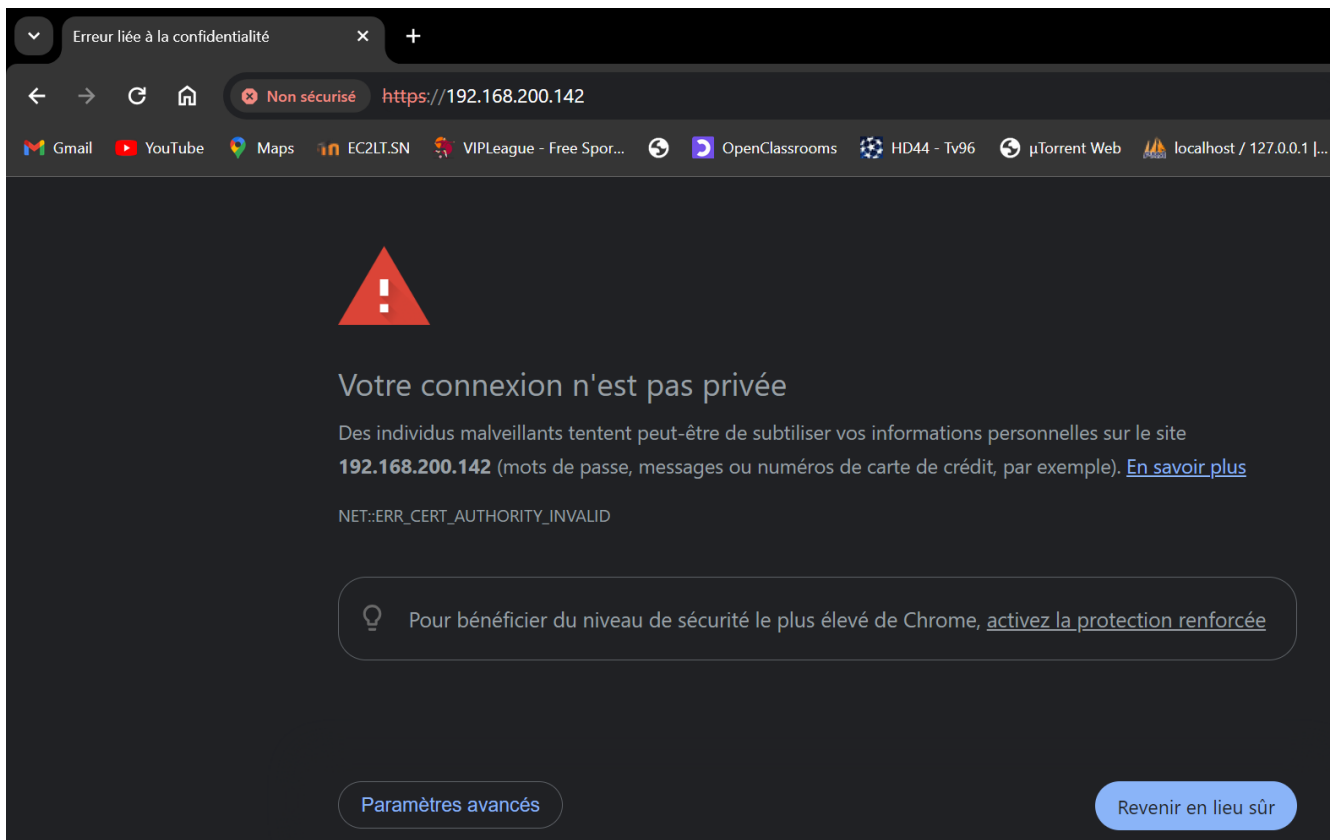
L'erreur **ssl_error_bad_cert_domain** signifie que le nom de domaine dans le certificat SSL ne correspond pas au nom de domaine auquel on essaie d'accéder. En d'autres termes, le certificat SSL est émis pour un domaine spécifique qui est localhost, mais nous essayons d'accéder au site via un autre domaine qui est 127.0.0.1.

16. Pourquoi voit-on sur le cadenas le message : Votre connexion à ce site n'est pas sécurisée ?



Le navigateur affiche un avertissement indiquant que la connexion n'est pas sécurisée car le certificat SSL ne correspond pas au domaine spécifié dans l'URL.

17. Utilisons une machine Windows pour accéder à notre site :

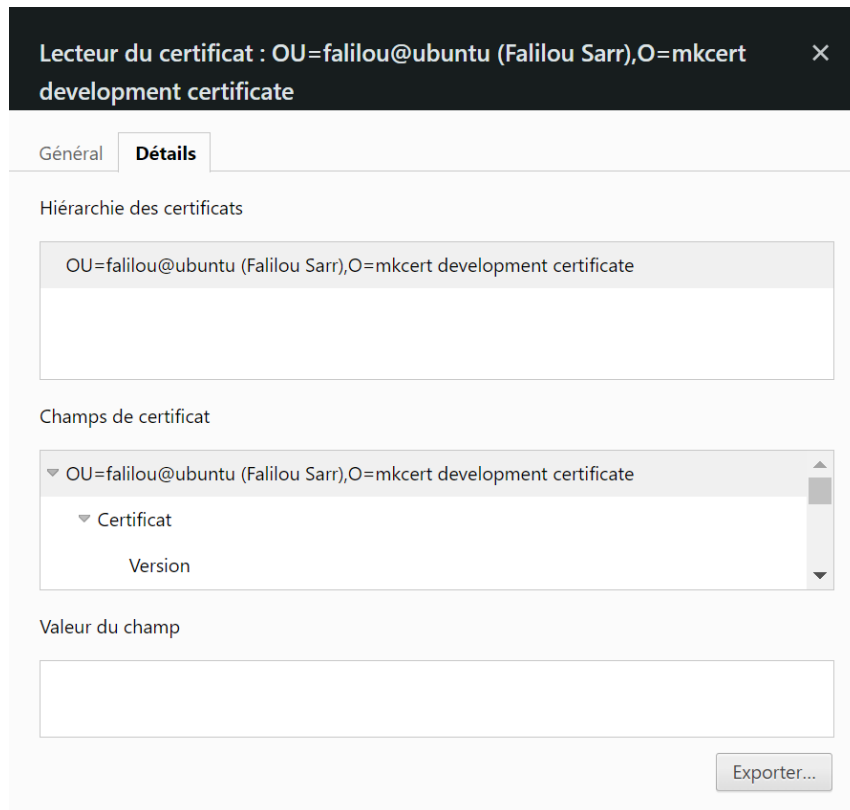


Bienvenue sur le site sécurisé de Falilou Sarr

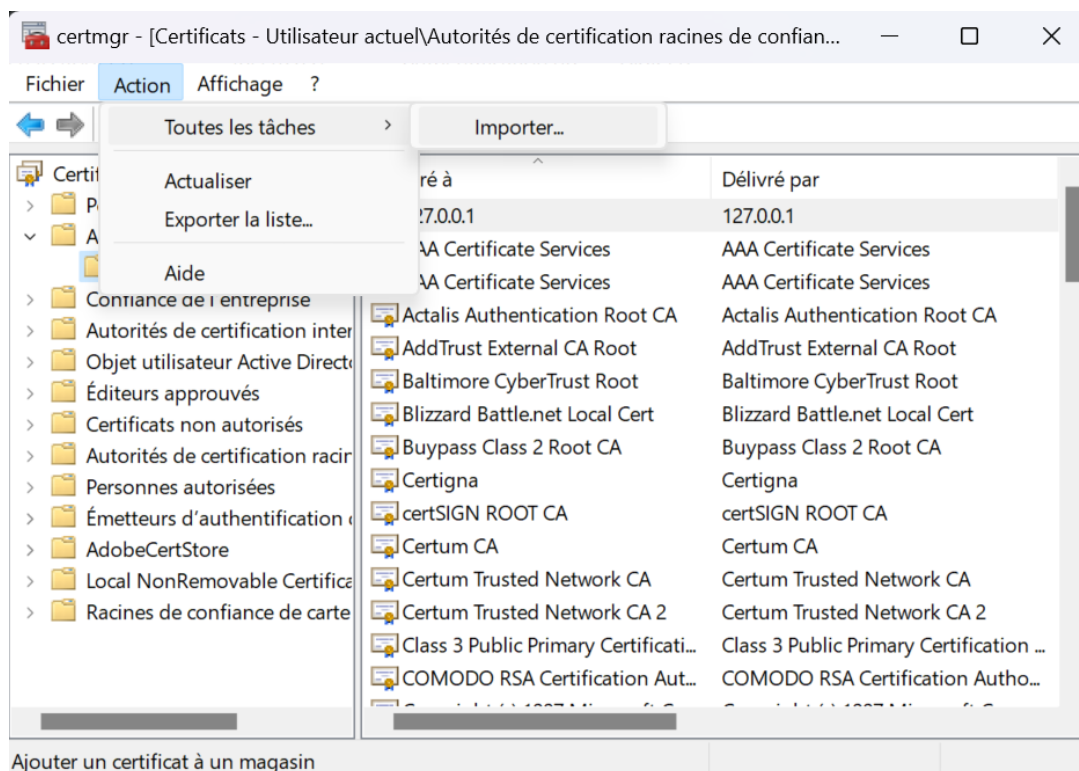
Ce site est sécurisé grâce à TLS (Transport Layer Security).

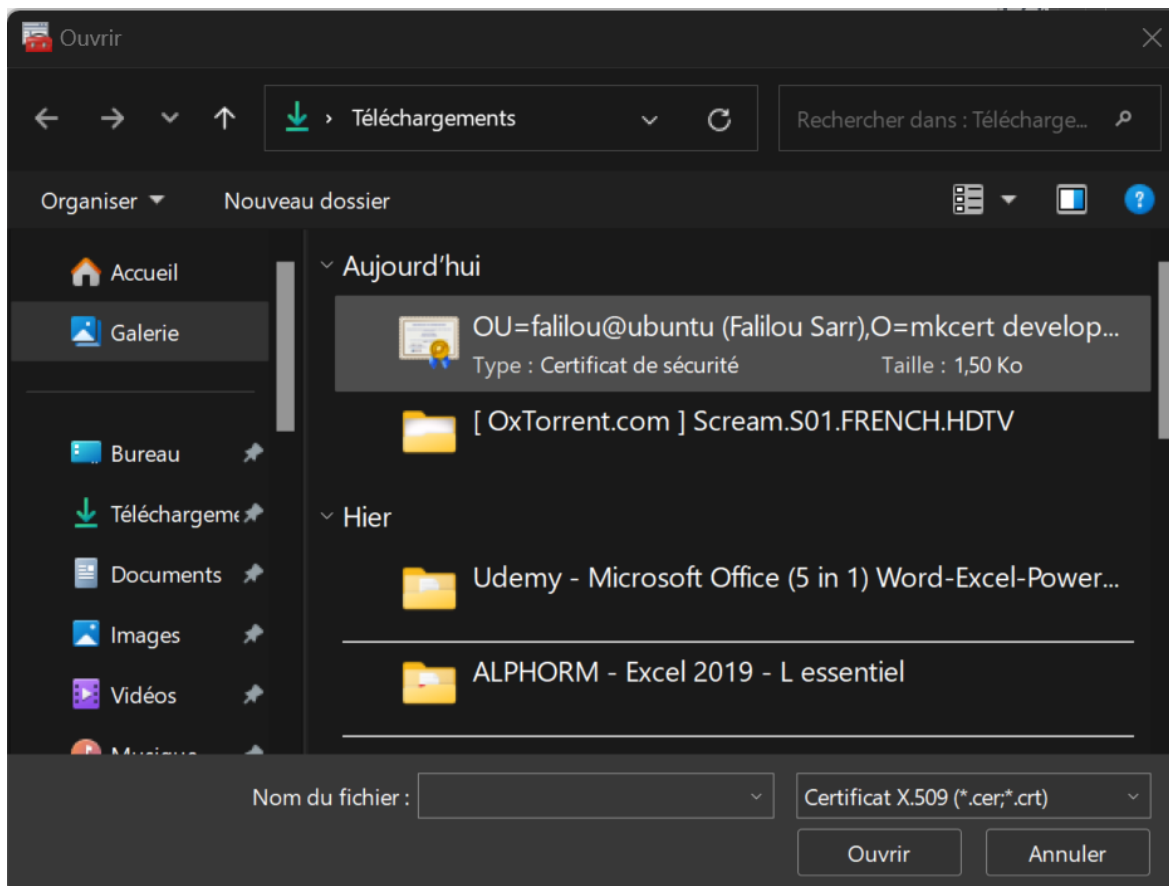
18. Une exception de sécurité liée à l'absence de l'autorité de confiance.
Ajoutons le certificat de l'autorité dans le magasin des autorités de certification racines de confiance :

 Exportons le certificat :

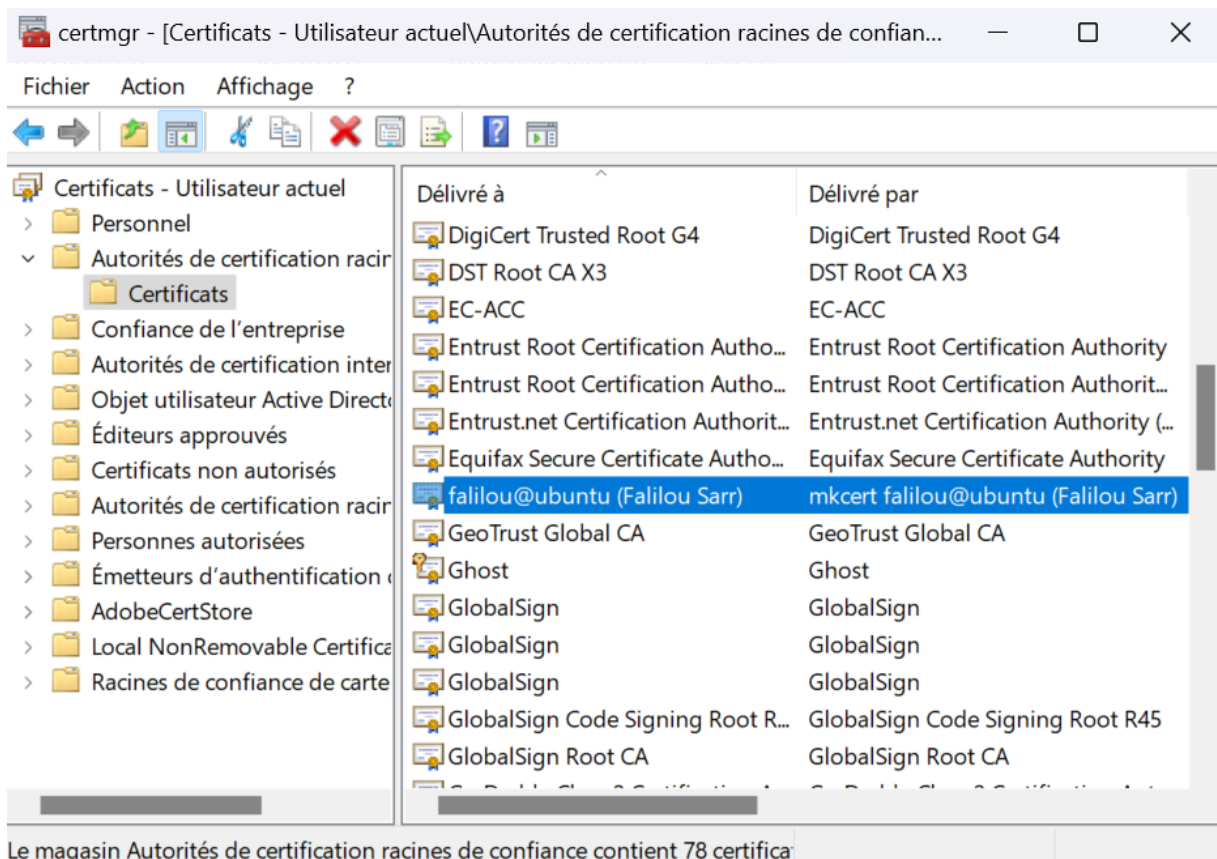


 Ajoutons le certificat dans le magasin des autorités de certification racines de confiance :

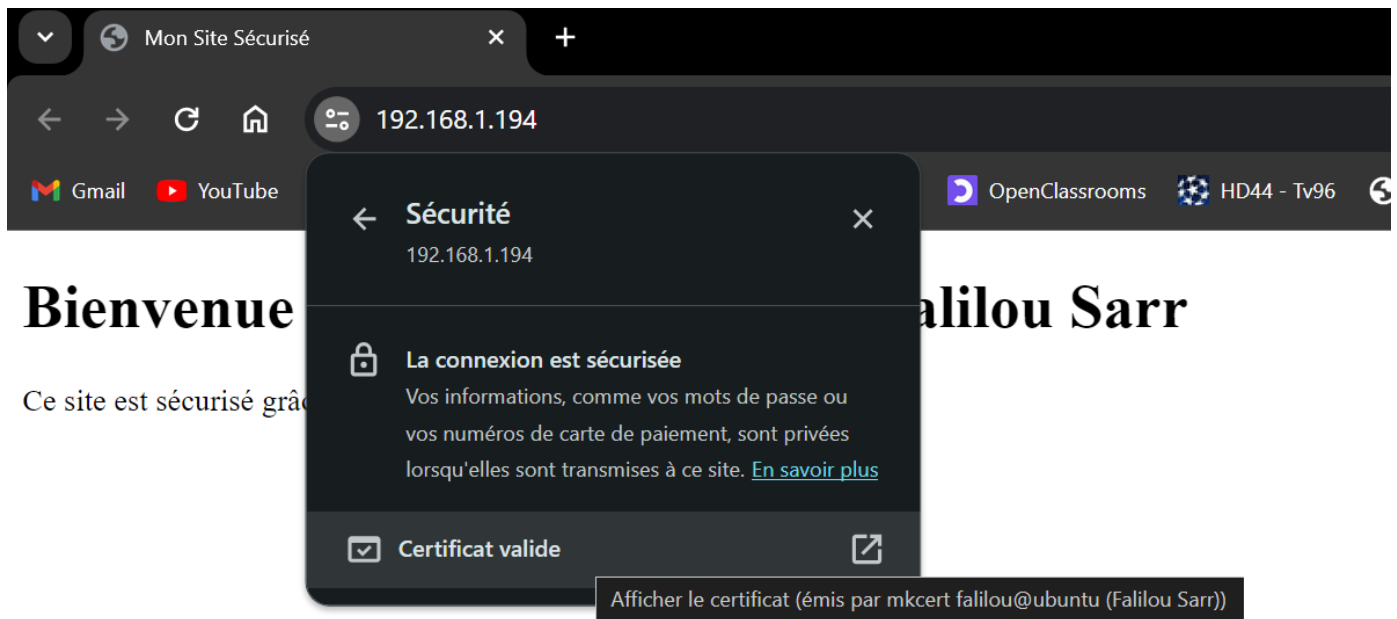




Notre certificat est ajouté dans le magasin de certification :



18. Redémarrons le navigateur et donnons nos remarques :



Une fois que vous avez importé avec succès le certificat de l'autorité locale dans le magasin des autorités de certification racines de confiance de Windows et redémarré votre navigateur, on doit pouvoir accéder à notre site sans rencontrer d'exception de sécurité liée à l'absence de l'autorité de confiance. Cela signifie que le certificat émis par notre autorité locale est maintenant considéré comme valide et fiable par notre système d'exploitation et notre navigateur, ce qui permet d'établir une connexion HTTPS sécurisée sans afficher d'avertissements ou d'erreurs de certificat.

19. Faire un mini programme en PHP avec une page de connexion qui après authentification autorise ou refuse l'accès :

 Créons la base de données où l'on stockera les utilisateurs :

```
mysql> create database auth;
Query OK, 1 row affected (0,04 sec)

mysql> 
```

 Créons la table et insérons des enregistrements :

```
mysql> create table users(id int primary key auto_increment, username varchar(30), password varchar(30));
Query OK, 0 rows affected (0,17 sec)

mysql> insert into users(username,password) values("falilou","passer");
Query OK, 1 row affected (0,03 sec)

mysql> insert into users(username,password) values("ghost","passer");
Query OK, 1 row affected (0,03 sec)

mysql> 
```

 Affichons les enregistrements de la table users :

```
mysql> select * from users;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1 | falilou | passer |
| 2 | ghost   | passer |
+----+-----+-----+
2 rows in set (0,00 sec)

mysql> 
```

 Donnons tous les privilèges à la base de données à l'utilisateur falilou :

```
mysql> grant all privileges on auth to falilou;
Query OK, 0 rows affected (0,02 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0,01 sec)

mysql> 
```

Ecrivons le code php pour la page d'authentification qui utilisera la base données auth pour vérifier les utilisateurs. Si l'utilisateur est dans la base de données le site affiche « vous pouvez accéder au site » sinon il affiche « Vous ne pouvez pas accéder au site » :

```
GNU nano 4.8 auth.php
<?php
$servername = "localhost";
$username = "falilou";
$password = "passer";
$dbname = "auth";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Erreur de connexion à la base de données : " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $input_username = $_POST["username"];
    $input_password = $_POST["password"];

    $sql = "select * from users where username = '$input_username' and password = '$input_password'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        echo "Vous pouvez accéder au site.";
    } else {
        echo "Vous ne pouvez pas accéder au site.";
    }
}
?>

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Page de Connexion</title>
</head>
<body>
    <h2>Connexion</h2>
    <form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>">
        <div>
            <label for="username">Nom d'utilisateur:</label>
            <input type="text" id="username" name="username" required>
        </div>
        <div>
            <label for="password">Mot de passe:</label>
            <input type="password" id="password" name="password" required>
        </div>
        <div>
            <button type="submit">Connexion</button>
        </div>
    </form>
</body>
</html>
```

Accédons au site d'authentification avec localhost/auth.php :

Page de Connexion

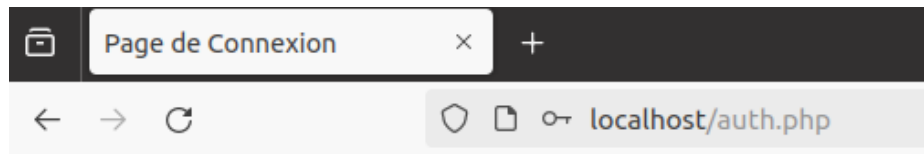
localhost/auth.php

Connexion

Nom d'utilisateur:

Mot de passe:

 Connectons-nous avec falilou qui se trouve dans la base de données :



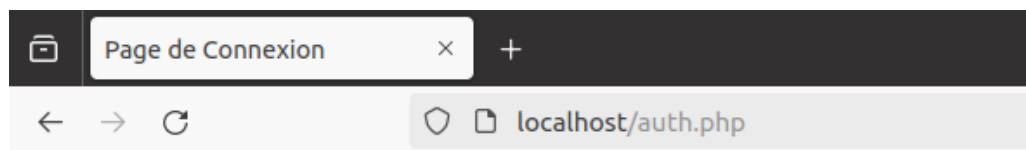
Vous pouvez accéder au site.

Connexion

Nom d'utilisateur:

Mot de passe:

 Connectons-nous avec un utilisateur au hasard qui ne se trouve pas dans la base de données :



Vous ne pouvez pas accéder au site.

Connexion

Nom d'utilisateur:

Mot de passe:

20. Sécurisons le site d'authentification :

 Copions notre configuration précédente en auth.conf :

```
root@ubuntu:/etc/apache2/sites-available# cp falilou-ssl.conf auth.conf
root@ubuntu:/etc/apache2/sites-available#
```

Ajoutons notre fichier index dans la configuration :

```
GNU nano 4.8 auth.conf
<IfModule mod_ssl.c>
  <VirtualHost _default_:443>
    ServerAdmin webmaster@localhost
    ServerName localhost
    DocumentRoot /var/www/html
    DirectoryIndex auth.php

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    SSLEngine on
    SSLCertificateFile      /home/falilou/localhost+1.pem
    SSLCertificateKeyFile   /home/falilou/localhost+1-key.pem

    <FilesMatch "\.(cgi|shtml|phtml|php)$">
      SSLOptions +StdEnvVars
    </FilesMatch>
    <Directory /usr/lib/cgi-bin>
      SSLOptions +StdEnvVars
    </Directory>
  </VirtualHost>
</IfModule>
```

Activons le site :

```
root@ubuntu:/etc/apache2/sites-available# a2ensite auth.conf
Enabling site auth.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@ubuntu:/etc/apache2/sites-available# systemctl reload apache2
root@ubuntu:/etc/apache2/sites-available#
```

Accédons au site avec https :

Page de Connexion

← → ↻ https://192.168.1.194/auth.php

Connexion

Nom d'utilisateur:

Mot de passe:

Connectons-nous avec mariam :

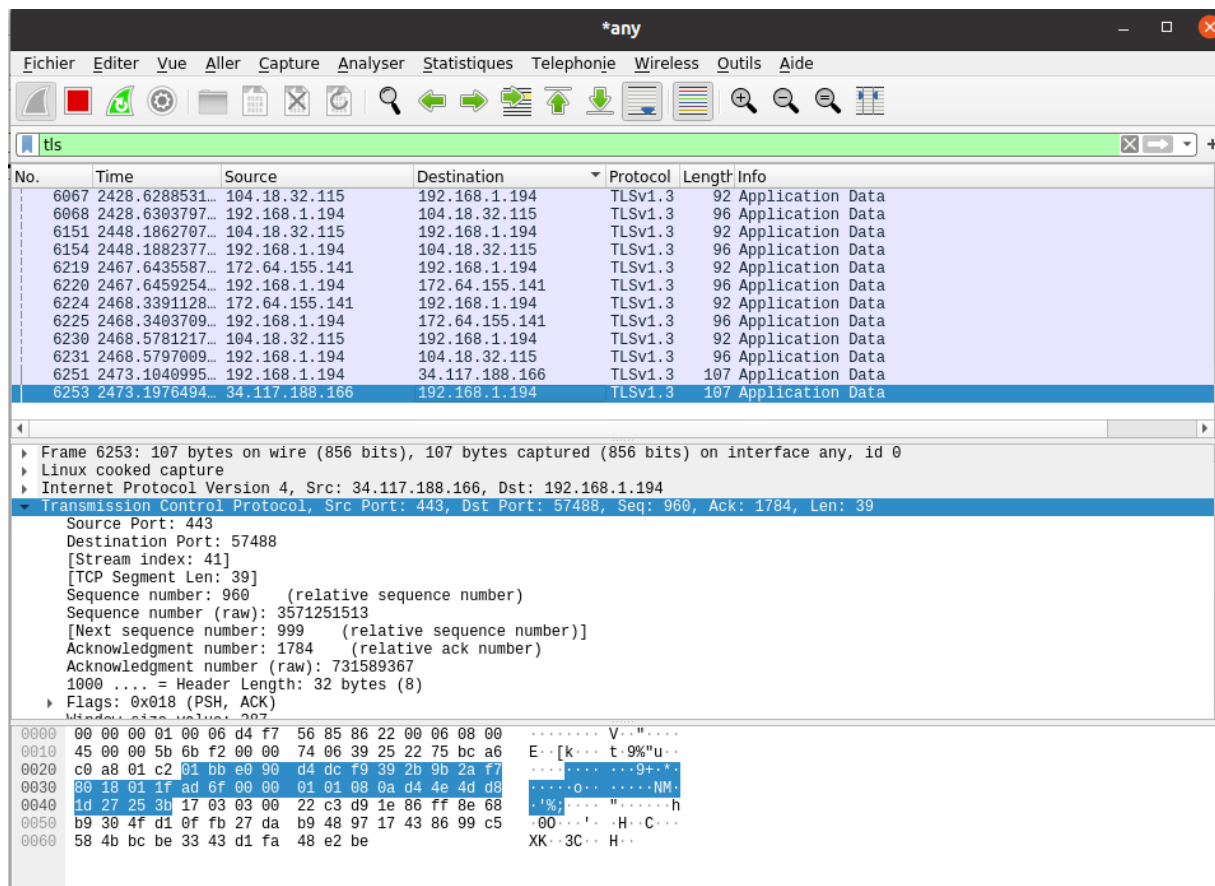
Page de Connexion

← → ↻ https://192.168.1.194/auth.php

Connexion

Nom d'utilisateur:

Mot de passe:



The image shows a Wireshark capture of network traffic on an interface named 'any'. The capture is filtered for 'tls' and shows a list of packets. The selected packet is frame 6253, which is a TLSv1.3 Application Data packet. The packet details pane shows the following information:

- Frame 6253: 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interface any, id 0
- Linux cooked capture
- Internet Protocol Version 4, Src: 34.117.188.166, Dst: 192.168.1.194
- Transmission Control Protocol, Src Port: 443, Dst Port: 57488, Seq: 960, Ack: 1784, Len: 39
 - Source Port: 443
 - Destination Port: 57488
 - [Stream index: 41]
 - [TCP Segment Len: 39]
 - Sequence number: 960 (relative sequence number)
 - Sequence number (raw): 3571251513
 - [Next sequence number: 999 (relative sequence number)]
 - Acknowledgment number: 1784 (relative ack number)
 - Acknowledgment number (raw): 731589367
 - 1000 = Header Length: 32 bytes (8)
 - Flags: 0x018 (PSH, ACK)

The packet bytes pane shows the raw data of the TLSv1.3 Application Data packet, which is encrypted. The data is displayed in hexadecimal and ASCII format, showing a mix of random-looking bytes and some recognizable characters like 'V', 'E', 't', '9', 'u', '9+', 'NM', 'h', 'H', 'C', 'H', 'XK', '3C', 'H'.